



Alma Mater Studiorum-Università di Bologna Scuola di Ingegneria

Dai Linguaggi alle Infrastrutture Software

Corso di Laurea in Ingegneria Informatica
Anno accademico 2023/2024

Prof. ENRICO DENTI

Dipartimento di Informatica – Scienza e Ingegneria (DISI)



“PROGRAMMARE” ... ?

- In passato, “occuparsi di informatica” è stato a lungo sinonimo di “programmare computer”
- Fa pensare a una *attività poco stimolante*, dove le *fasi creative – analisi e progetto – sono già avvenute*
 - tipico della costruzione di *algoritmi* di medio/piccola dimensione, normalmente già noti e descritti in letteratura
- Anni '80: “crisi del software”
insufficienza di metodologie e strumenti nati per una *visione “algoritmica” dell'informatica (programmazione in-the-small)* rispetto a progettazione, sviluppo e manutenzione di *sistemi software complessi (progettazione in the large)*



PROGETTARE SISTEMI SOFTWARE

- Il termine “programmare” richiama una *visione obsoleta* dell'informatica
- Nuovo obiettivo:
saper progettare e costruire sistemi software
con il *rigore* e le *garanzie* che si richiedono
a tutti gli altri prodotti dell'ingegneria
 - qualità
 - affidabilità

Necessità di *uscire dall'artigianato*

Concepire il *software* come un *prodotto industriale*

Prossimamente: usare *anche l'AI* ma *dominandola*



UN CAMBIO DI PARADIGMA

- Uscire dall'artigianato

- non reinventare la ruota ogni giorno
- impossibile fare le cose da soli "nel proprio sottoscala"
- pieno supporto al *lavoro di squadra*

- Software come prodotto industriale

- strutturazione precisa del *processo di produzione*
- supporto al *versioning* del software
- *collaudo sistematico* come parte integrante del processo
- *automatizzazione* degli step chiave del processo
- cura degli *aspetti legali*: *privacy by design and by default*
(General Data Protection Regulation – GDPR, 25/5/2018)



GESTIRE IL PROCESSO DI SVILUPPO

- Scrivere il software è solo una parte del problema
- Gestire il *processo di sviluppo* è altrettanto importante, nonché *critico e costoso*
- *Un aspetto critico: il controllo di versione*
 - lo sviluppo del software passa attraverso *molti file*
 - si fanno modifiche, poi si torna indietro, ci si pente
 - più persone lavorano contemporaneamente su più parti
 - pensare di farlo col *copia & incolla* significa *avviarsi al disastro*
 - non si sa più cosa c'è in ogni file
 - non si riescono a mantenere coerenti versioni successive
 - grande spreco di tempo e di soldi
 - occorre affidarsi a un *sistema di versioning*



SISTEMI DI VERSIONING

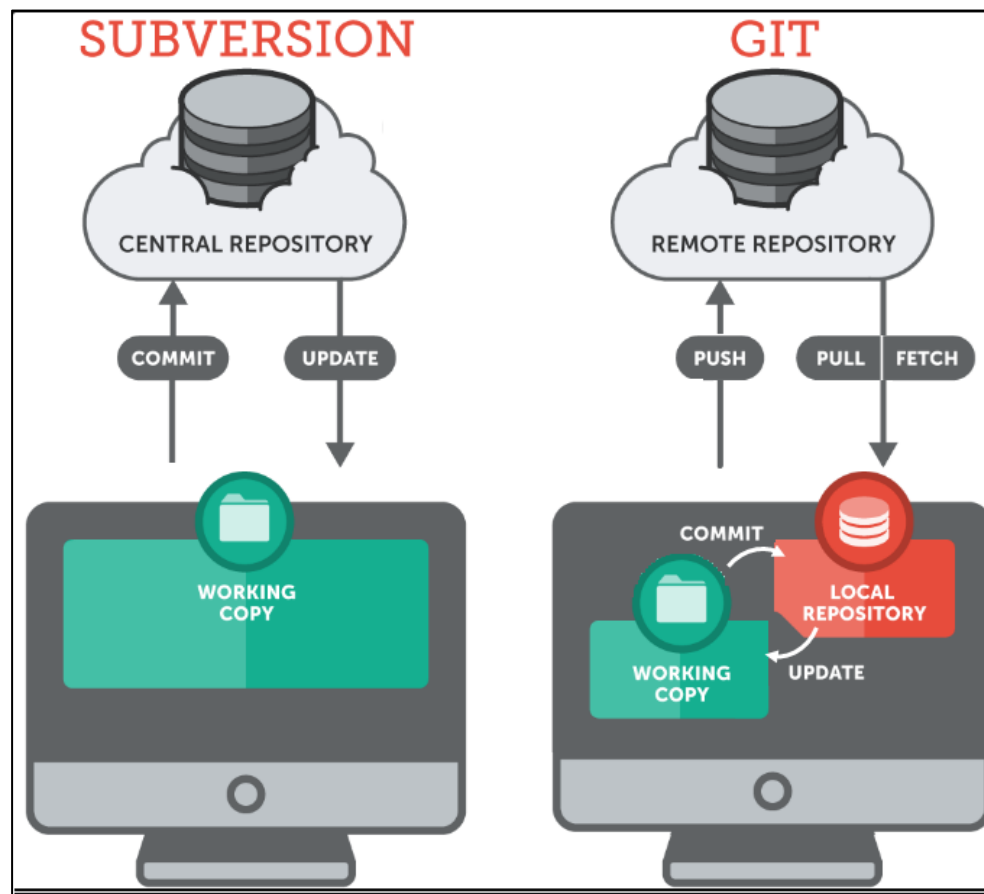
- Un **sistema di versioning** permette di:
 - **tenere traccia di ogni versione di ogni singolo file** del progetto
 - **tornare indietro** a versioni precedenti, senza mai perdere niente anche in caso di sbagli
 - **gestire diramazioni (branch)** sperimentali di un software, senza "pasticciare" sulla versione principale
 - **riunire (merge) file** di versioni diverse in una versione unica
 - **ottenere report** sulle differenze fra file di versioni diverse
 - **gestire le dipendenze** fra progetti diversi o fra versioni per piattaforme diverse
 - **supportare il lavoro di squadra** (chi ha fatto cosa e quando), anche nel distribuito, anche offline

SISTEMI DI VERSIONING

- I sistemi di versioning più diffusi sono **SVN** e **GIT**

- Idea base

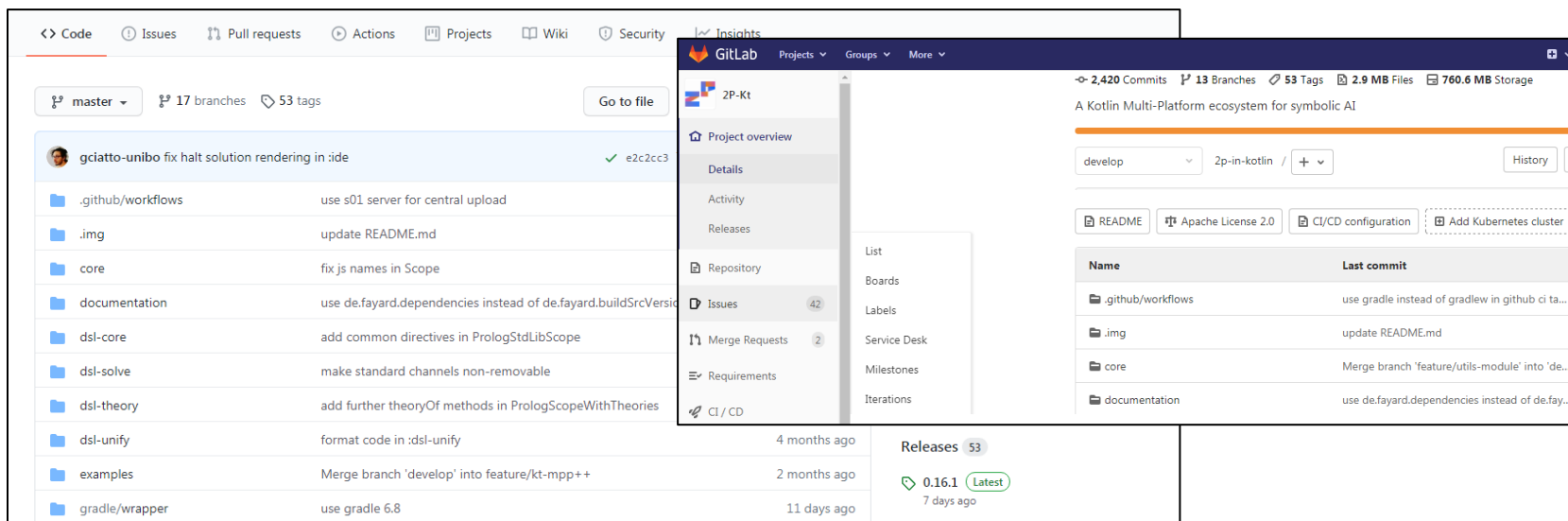
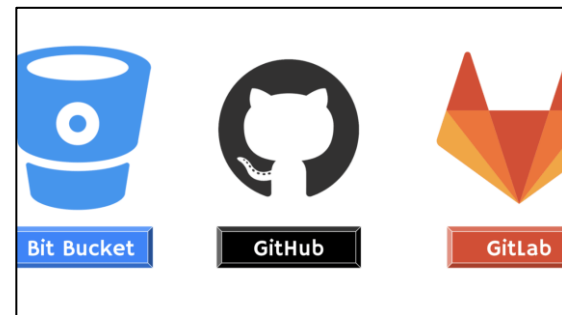
- i file del progetto locale sono copiati, a richiesta, su un **repository** che traccia le versioni
- in **SVN**, il repository è unico e **centralizzato**
- in **GIT**, ogni pc ha il suo repository (**distribuito**), che solo a richiesta è copiato nel **repository centrale remoto**
→ oggi più diffuso



SISTEMI DI VERSIONING

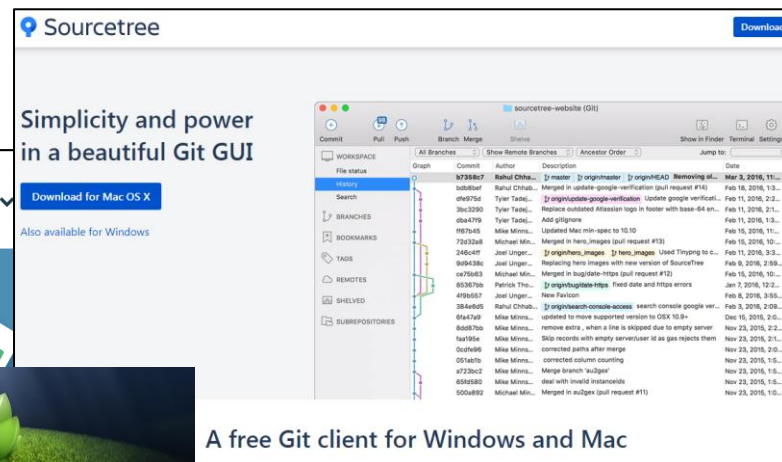
- Esistono molti siti che offrono *hosting* **SVN** e/o **GIT**

- per GIT,
i più noti sono GitHub, GitLab, BitBucket
- per SVN (oggi meno usato),
GitHub, RiouxSVN, Atlassian, etc.

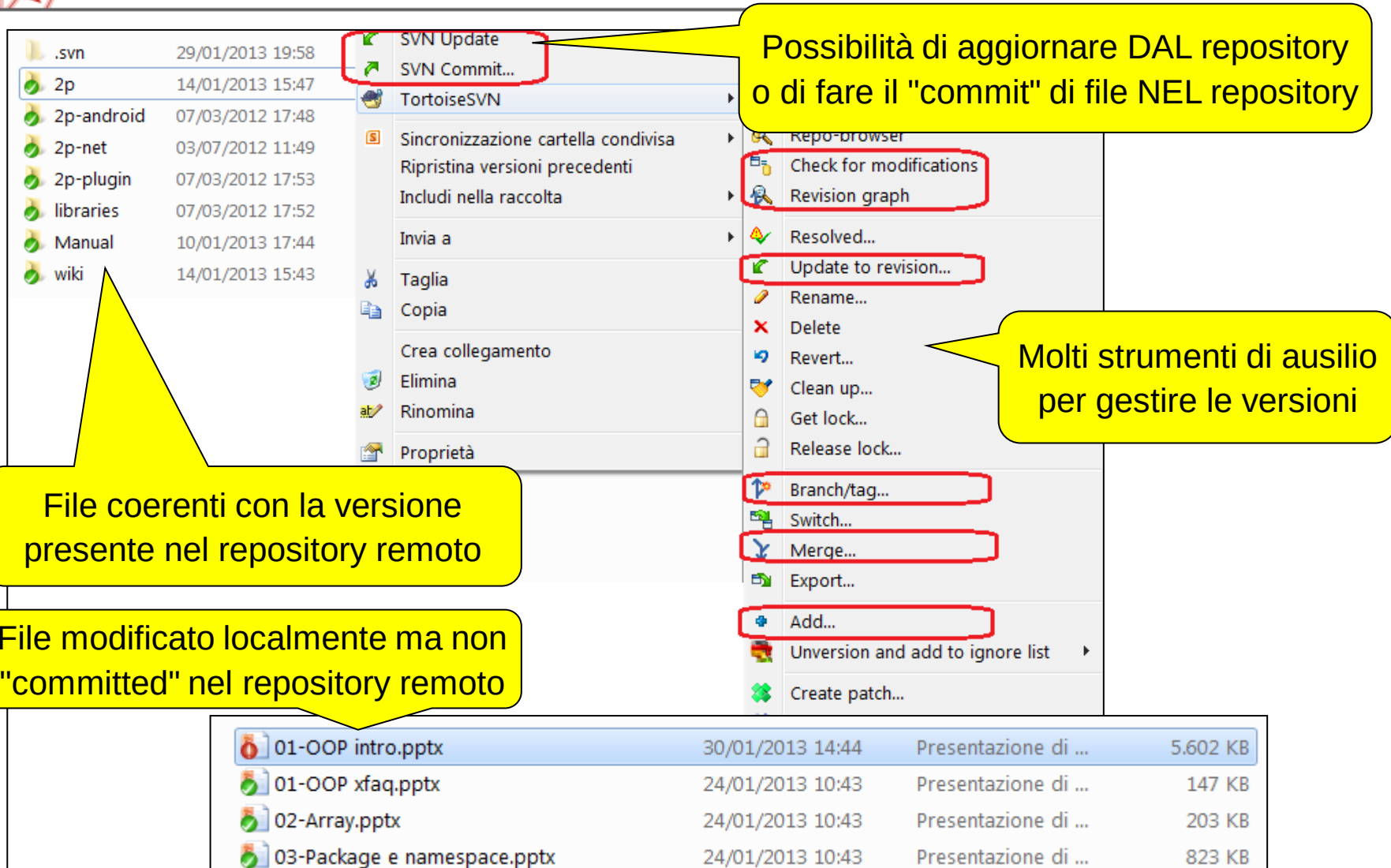


SISTEMI DI VERSIONING

- Serve poi un *client* sul proprio pc
 - Windows: TortoiseSVN, TortoiseGIT, SourceTree,...
 - Mac: SourceTree, Versions,...
 - spesso integrati negli strumenti (es. Eclipse, IntelliJ, ...)



ESEMPIO: Subversion (SVN)



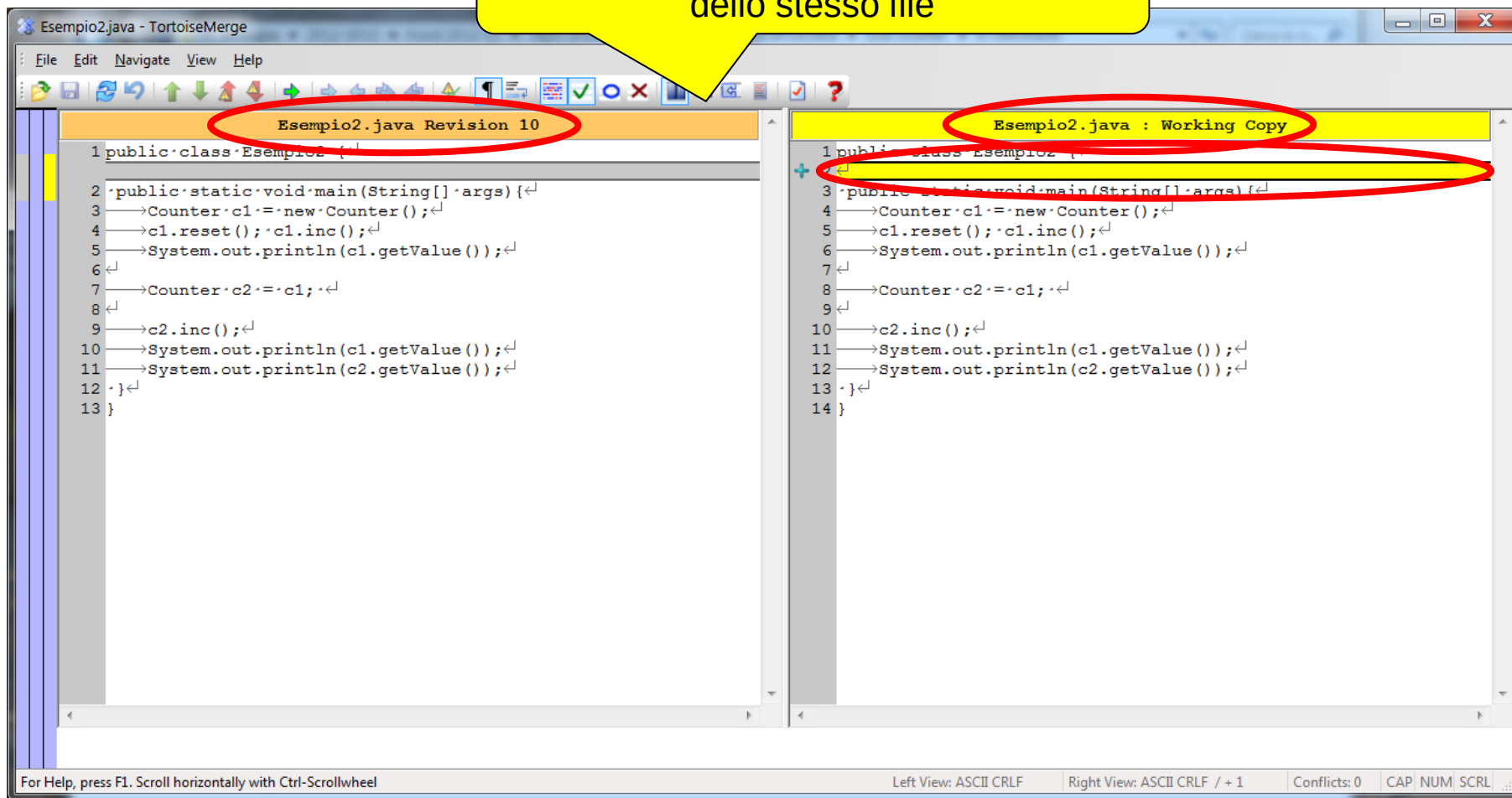
The screenshot shows the TortoiseSVN context menu and a file list. Annotations highlight key features:

- SVN Update** and **SVN Commit...** are highlighted in the menu, with a callout: "Possibilità di aggiornare DAL repository o di fare il 'commit' di file NEL repository".
- Check for modifications**, **Revision graph**, **Update to revision...**, **Branch/tag...**, **Switch...**, **Merge...**, and **Add...** are highlighted in the menu, with a callout: "Molti strumenti di ausilio per gestire le versioni".
- The **.svn** folder in the file list is highlighted, with a callout: "File coerenti con la versione presente nel repository remoto".
- The **2p** file in the file list is highlighted, with a callout: "File modificato localmente ma non 'committed' nel repository remoto".

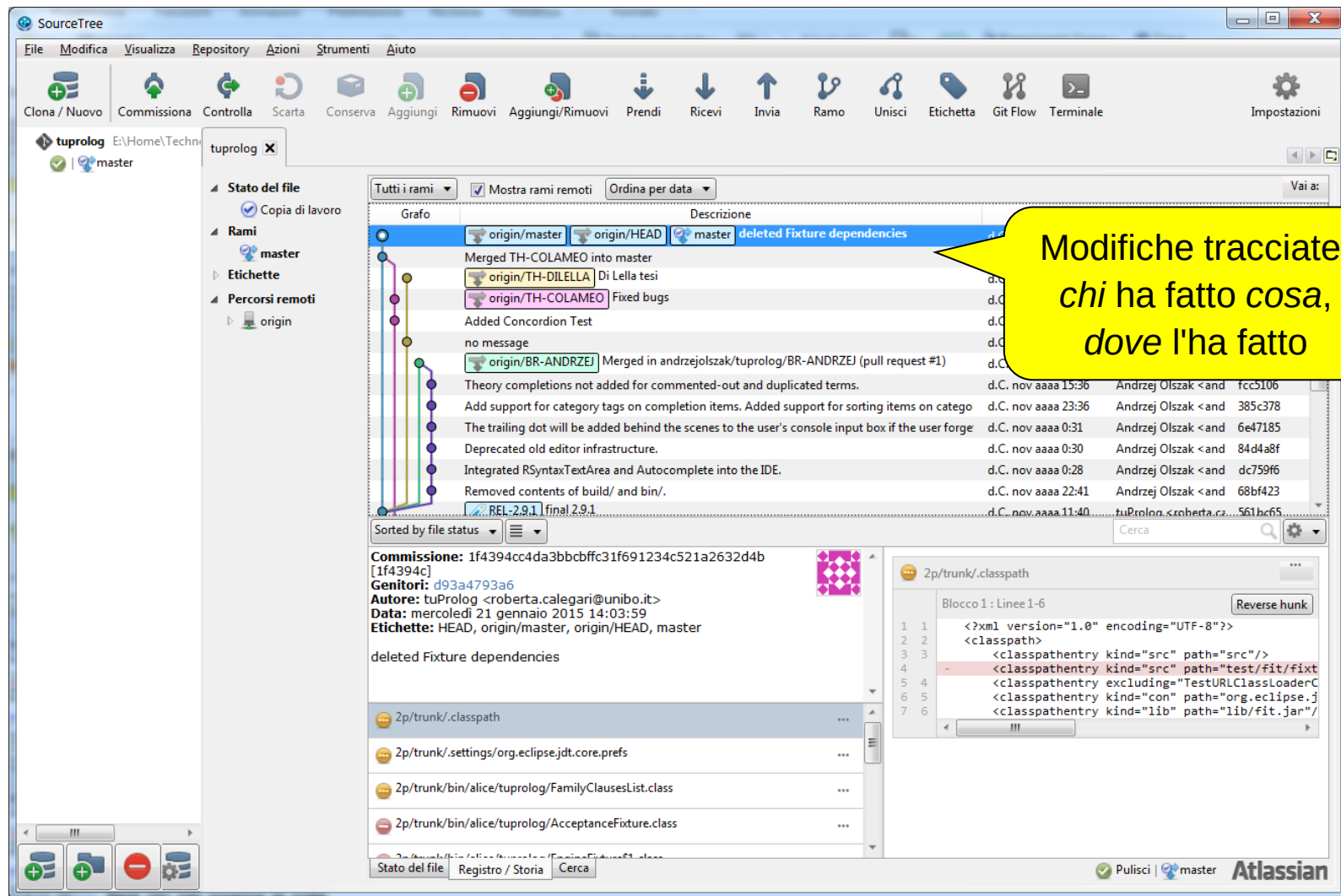
File Name	Date	Description	Size
01-OOP intro.pptx	30/01/2013 14:44	Presentazione di ...	5.602 KB
01-OOP xfaq.pptx	24/01/2013 10:43	Presentazione di ...	147 KB
02-Array.pptx	24/01/2013 10:43	Presentazione di ...	203 KB
03-Package e namespace.pptx	24/01/2013 10:43	Presentazione di ...	823 KB

ESEMPIO: Subversion (SVN)

Differenze fra due versioni successive
dello stesso file



ESEMPIO: Git



SourceTree

File Modifica Visualizza Repository Azioni Strumenti Aiuto

Clona / Nuovo Commissiona Controlla Scarta Conserva Aggiungi Rimuovi Aggiungi/Rimuovi Prendi Ricevi Invia Ramo Unisci Etichetta Git Flow Terminale Impostazioni

tuprolog E:\Home\Techni
master

Stato del file
Copia di lavoro

Rami
master

Etichette

Percorsi remoti
origin

Tutti i rami Mostra rami remoti Ordina per data

Grafo

Descrizione

origin/master origin/HEAD master deleted Fixture dependencies

Merged TH-COLAMEO into master

origin/TH-DILELLA Di Lella tesi

origin/TH-COLAMEO Fixed bugs

Added Concordion Test

no message

origin/BR-ANDRZEJ Merged in andrzejolszak/tuprolog/BR-ANDRZEJ (pull request #1)

Theory completions not added for commented-out and duplicated terms.

Add support for category tags on completion items. Added support for sorting items on catego

The trailing dot will be added behind the scenes to the user's console input box if the user force

Deprecated old editor infrastructure.

Integrated RSyntaxTextArea and Autocomplete into the IDE.

Removed contents of build/ and bin/.

REL-2.9.1 final 2.9.1

Sorted by file status

Commissione: 1f4394cc4da3bbcbffc31f691234c521a2632d4b [1f4394c]
Genitori: d93a4793a6
Autore: tuProlog <roberta.calegari@unibo.it>
Data: mercoledì 21 gennaio 2015 14:03:59
Etichette: HEAD, origin/master, origin/HEAD, master

deleted Fixture dependencies

2p/trunk/.classpath

2p/trunk/.settings/org.eclipse.jdt.core.prefs

2p/trunk/bin/alice/tuprolog/FamilyClausesList.class

2p/trunk/bin/alice/tuprolog/AcceptanceFixture.class

2p/trunk/bin/fit/fit/fit/FitTest.class

Stato del file Registro / Storia Cerca

2p/trunk/.classpath

Blocco 1: Linee 1-6

Reverse hunk

```

1 1 <?xml version="1.0" encoding="UTF-8"?>
2 2 <classpath>
3 3   <classpathentry kind="src" path="src"/>
4 4   <classpathentry kind="src" path="test/fit/fixt
5 5   <classpathentry excluding="TestURLClassLoaderC
6 6   <classpathentry kind="con" path="org.eclipse.j
7 7   <classpathentry kind="lib" path="lib/fit.jar"/

```

Pulisci master Atlassian



PROGETTARE (PER) IL COLLAUDO

- Le garanzie di qualità di un prodotto dell'ingegneria richiedono una **attività di collaudo *integrata e sistematica***

IL COLLAUDO

- **non si improvvisa**: va ***accuratamente progettato***
 - non dopo: *prima ancora* di costruire il sistema
- non è un'attività accessoria: ***è parte integrante del TUO lavoro***
- come tale, va ***strumentato e supportato*** in modo completo
- è espresso dal *piano di collaudo*
 - il piano *guida la validazione* del sistema
- non è fare "qualche stampa" (chi le legge? se sono centinaia?)

TIPI DI COLLAUDO

Esistono diversi *approcci*, con fini diversi.

BLACK-BOX TESTING

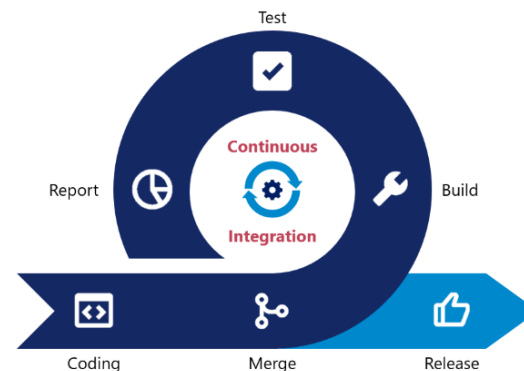
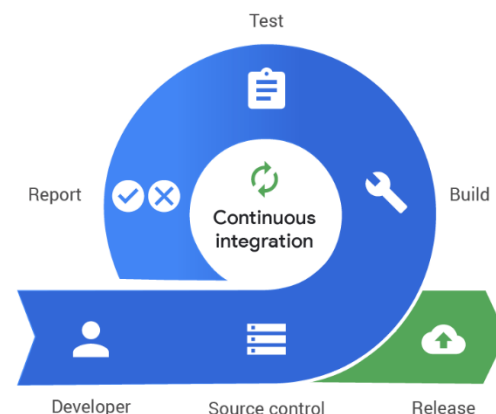
- collaudo **progettato e svolto "ai morsetti"** di un componente software, ***senza sapere com'è fatto dentro***
- scopo: verificare la correttezza del comportamento esterno, indipendentemente dalla realizzazione

WHITE-BOX TESTING

- collaudo **progettato e svolto *tenendo esplicitamente in conto com'è fatto dentro*** un componente
- scopo: verificare la corretta implementazione delle varie funzioni, *stressandole una ad una* anche nei casi critici

CONTINUOUS INTEGRATION

- Il collaudo è parte integrante della strategia di sviluppo, oggi molto diffusa, detta *Continuous Integration*
 - il codice è sotto controllo di versione
 - a ogni modifica, si ri-eseguono i test e si ottengono report dettagliati
 - solo se OK, la modifica è approvata; altrimenti, si corregge o si ripristina la versione precedente
 - di tanto in tanto, si rilascia una versione ritenuta consistente (*release*)
- Tutto è sempre sotto controllo





L'IMPORTANZA DEL COLLAUDO SISTEMATICO

- Ad esempio, @Google....
[Communications of the ACM, 2018(4)]

Testing. Nearly all Google code includes corresponding tests, ranging from unit tests all the way to large-scale integration tests. Tests are integrated as a first-class concept in the build system and hermetic and distributed, just like builds. For most projects, developers write and maintain the tests for their code; projects typically have no separate testing or quality-assurance group. Google's continuous build-and-test system runs tests on every commit and notifies a developer if the developer's change broke the build or caused a test to fail. It also supports testing a change before committing to avoid breaking downstream projects.

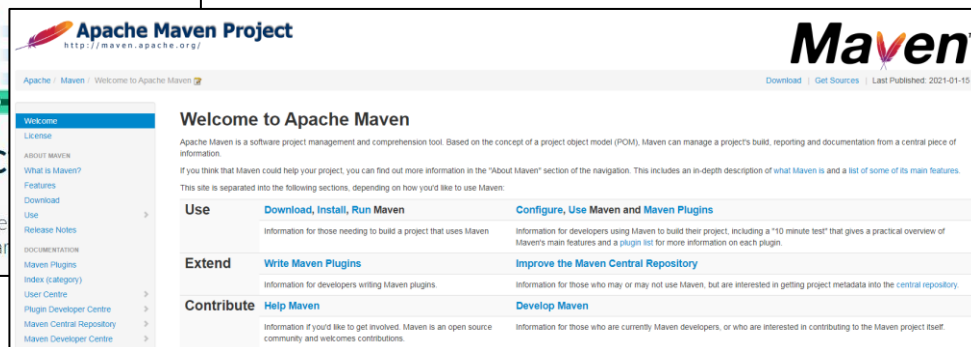
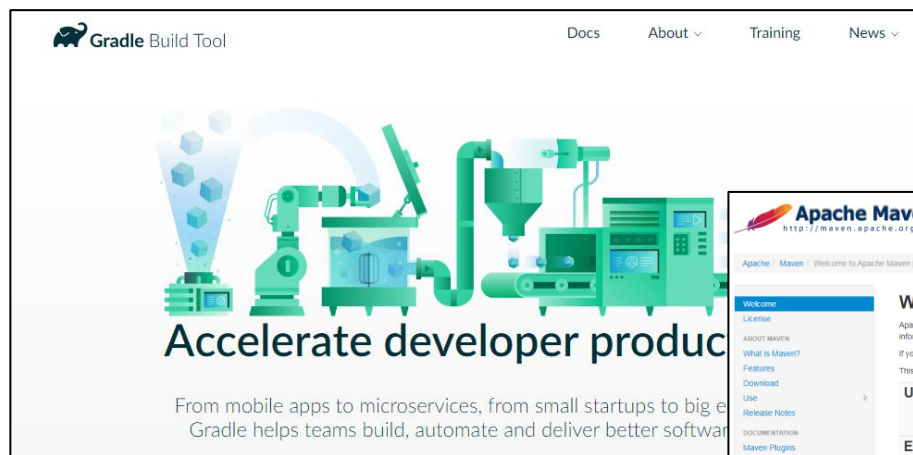
Criteria for compiler checks. To scale-up our work, we have defined criteria for enabling checks in the compiler, setting the bar high, since breaking the compile would be a significant disruption. A

Gating commits. Some teams want specific analyzers to actually block commits, rather than just appear in the code-review tool. The ability to block commits is commonly requested by teams that have highly specific custom checks with no false positives, usually for a custom DSL or library; and

Analysing code before building. We advocate for a system focused on pushing workflow integration as early as possible. When possible, checks are enabled as compiler errors. To avoid breaking builds, tool writers take on the task of first fixing all the existing issues in the codebase, allowing us to "ratchet" the quality of Google's codebase one small step at a time, without regressions. Since we present the errors in the compiler, developers encounter them immediately after writing code, while they are still amenable to making changes. To enable this,

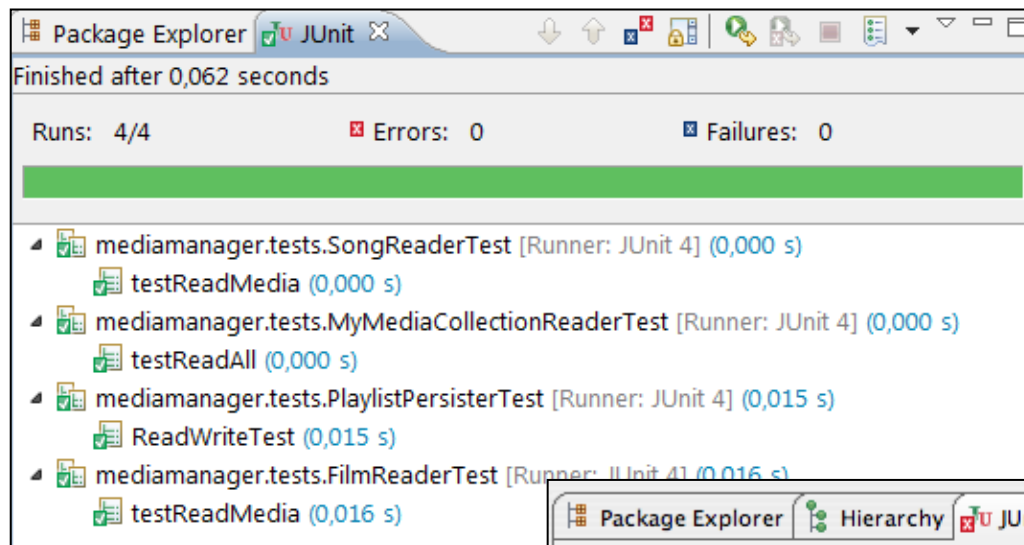
STRUMENTARE IL PROCESSO

- Per questo esistono strumenti che gestiscono / automatizzano il ciclo di sviluppi: i cosiddetti *Build tools*
- Esempi: Gradle, Maven, ...
 - in Gradle si specifica, con un apposito linguaggio, il grafo delle dipendenze fra più progetti + il relativo ciclo di sviluppo



STRUMENTARE IL COLLAUDO

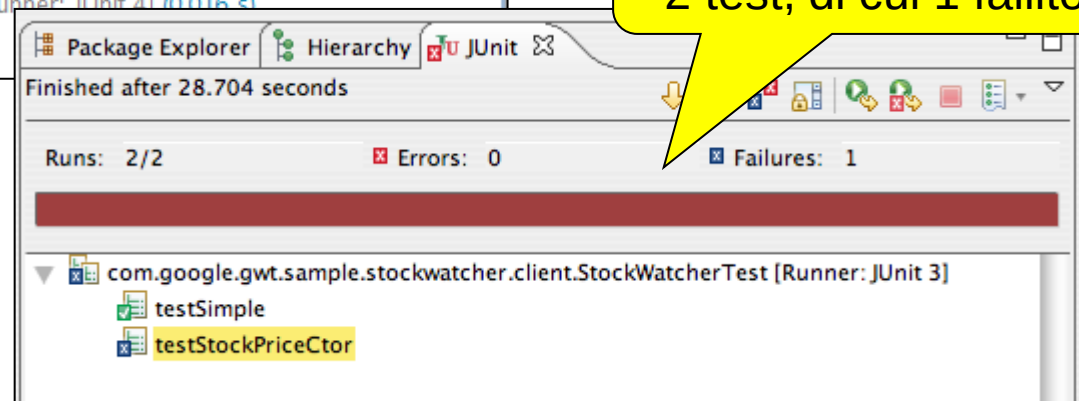
Per Java, lo strumento JUnit *esegue in automatico i test progettati*, riportandone i risultati in forma grafica



Esecuzione di un piano di collaudo: 4 test positivi su 4

A **failure** is when one of your assertions fails.

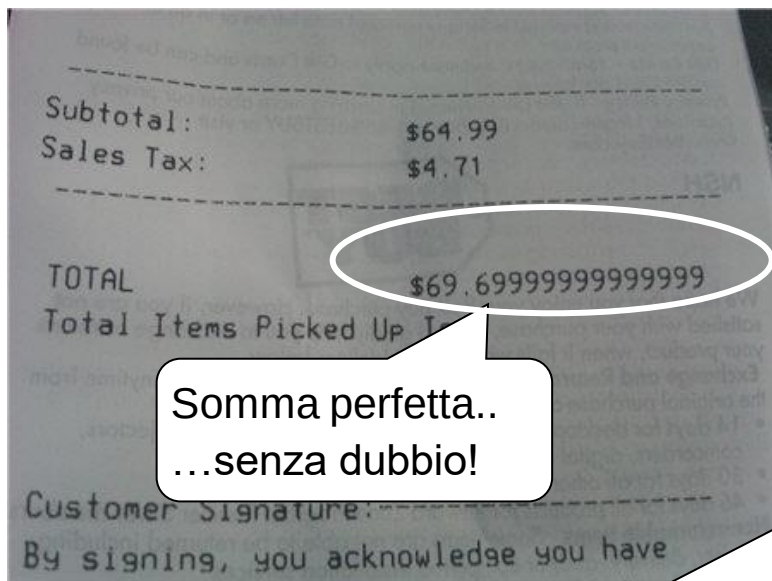
An **error** is when some other exception occurs, one you haven't tested for and didn't expect.



Controesempio: 2 test, di cui 1 fallito

COLLAUDARE SISTEMI SOFTWARE...?

- Se il software fosse collaudato con la stessa cura e lo stesso rigore che si richiedono per automobili, ascensori, case...
si eviterebbero "figuracce" (e danni) come questi



Price	Departure	Arrival	Travel time	Operated by	Flight number
EUR 574	06:45 Bologna 10:20 Amsterdam 16:17 Atlanta	09:05 Amsterdam 14:10 Atlanta 18:06 Miami	17h21		* Details
EUR 1,760	07:10 Bologna 09:20 Rome	08:10 Rome 14:45 Miami	13h35		* Details
EUR 1,711	07:15 Bologna 10:50 Paris (CDG)	09:00 Paris (CDG) 14:35 Miami	13h20		* Details
Please note: only 2 seat(s) available at this price					
Show all flights >					
Taxes, surcharges and booking fee included					
Next day Lowest price on this page					
Return Tuesday 22 February 2011					
Show flights All departure times					
EUR 616	13:32 Miami 17:40 Detroit	16:40 Detroit 07:45 Amsterdam	15h38		* Details
This flight arrives 1 day later. 09:20 Amsterdam 11:10 Bologna Please note: only 2 seat(s) available at this price					
EUR 401	17:46 Miami 21:35 Atlanta	19:49 Atlanta 11:50 Amsterdam	17h04		* Details
This flight arrives 1 day later. 15:00 Amsterdam 16:50 Bologna					
EUR 483	18:00 Miami 13:10 Paris (CDG)	08:20 Paris (CDG) 14:50 Bologna	14h50		* Details
Show all flights >					
Taxes, surcharges and booking fee included					
Next day Lowest price on this page					

COLLAUDARE SISTEMI SOFTWARE...?

- Se il software fosse collaudato con la stessa cura e lo stesso rigore che si usano per automobili, ascensori, ponti...
si eviterebbero "figuracce" (e danni) come questi

Leave	Thu, Feb 17	Hawaiian Airlines 451
	Depart: 11:50am Arrive: 7:20pm	Honolulu/Oahu, HI (HNL) Sydney, Australia (SYD)
	Non-stop	Economy 10hr 30min B
	 This is an overnight flight.	
Return	Sun, Feb 27	Hawaiian Airlines 452
	Depart: 9:20pm Arrive: 10:10am	Sydney, Australia (SYD) Honolulu/Oahu, HI (HNL)
	Non-stop	Economy 9hr 50min Boeing 767 View

Come? Parte a mezzogiorno, arriva alla sera.. ed è overnight?? Ignoranza + bug: l'algoritmo non tiene conto della linea del cambiamento di data.

IRRILEVANTE? aspetta di avere una coincidenza da prendere, e scoprire che è stata prenotata il 17 febbraio invece del 18..! Torni indietro nel tempo a prenderla?

La versione corretta

DEPARTING FLIGHT

Oahu - Honolulu (HNL) to Sydney, Australia (SYD)
Hawaiian Coach

FEB 17

HA451 Thursday, February 17, 2011 11:50 AM–
Friday, February 18, 2011 7:20 PM (next day)
Nonstop

RETURN FLIGHT

Sydney, Australia (SYD) to Oahu - Honolulu (HNL)
Hawaiian Coach

FEB 27

HA452 Sunday, February 27, 2011 9:20 PM–10:10 AM
Nonstop

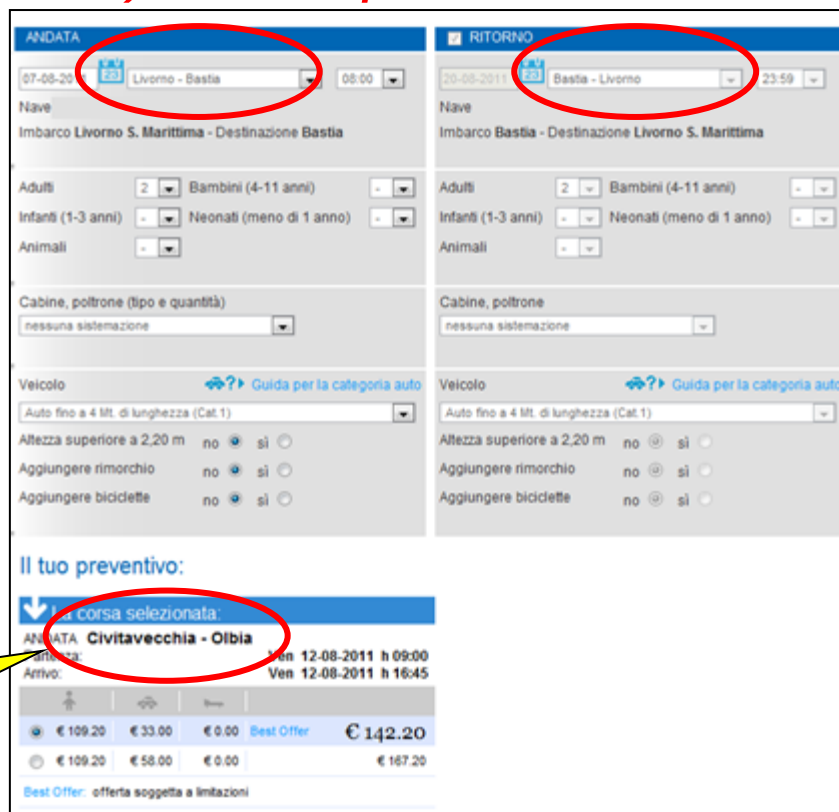
COLLAUDARE SISTEMI SOFTWARE...?

- Se il software fosse collaudato con la stessa cura e lo stesso rigore che si usano per automobili, ascensori, ponti...
si eviterebbero "figuracce" (e danni) come questi



Non male il treno "not found"..
Che provenga dal binario 9 ¾ ? ☺

Tu chiedi Livorno-Bastia A/R...
..e lui ti conferma un preventivo per
Civitavecchia-Olbia??



COLLAUDARE SISTEMI SOFTWARE...?

- Se il software fosse collaudato con la stessa cura e lo stesso rigore che si usano per automobili, ascensori, ponti...
si eviterebbero "figuracce" (e danni) come questi

DCI al 20/11/0731: 2.320

Vogliamo parlare della data 31/07/2011 formattata come GG/MM/AAAA ma coi dati invertiti, sugli estratti conto di decine di migliaia di clienti ?

E nessuno se ne accorge per mesi???

..per non dire di quando un'applicazione crolla rovinosamente e nessuno ha pensato alle conseguenze..



Destination	Flight Number	Gate	Time
08:30 Istanbul	EZY733	Gate opens	08:45
08:30 Madrid	EZY5477	Gate opens	08:45
08:30 Glasgow	BE9173	Gate opens	08:45
08:30 Montego Bay	TCX049K	Gate opens	08:30
09:45 Cagliari	MCX037	Gate opens	08:50
09:50 Florence	IG3536	Gate opens	08:15
10:00 Dublin	FR113	Gate opens	09:15
10:15 Shannon	BE911	Gate opens	09:30
10:30 Mexico City	MX1595	Gate opens	09:30
10:30 Sofia	FB496	Gate opens	09:15
10:30 Bridgetown	V5029	Gate opens	09:15
10:45 Las Vegas	V5043	Gate opens	09:30
10:45 Lisbon	TP345	Gate opens	09:45
10:55 Marseille	EZY5325	Gate opens	10:10
10:55 Valencia	EZY5149	Gate opens	10:10
10:55 Jersey	BE305	Gate opens	10:10
11:00 Charlotte	US733	Gate opens	10:10
11:00 Tripoli	BU913	Gate opens	10:10
11:15 Milan-Linate	EZY5285	Gate opens	10:30
11:15 Philadelphia	US731	Gate opens	09:45
11:15 Orlando	V5027	Gate opens	10:00
11:25 Accra	G0100	Gate opens	09:55
11:25 Chambéry	ZT2743	Gate opens	10:25
13:55 Milan-Malpensa	EZY5295	Gate opens	13:10
14:00 Budapest	EZY5443	Gate opens	13:15
14:00 Funkeventura	TCX432K	Gate opens	13:15
14:15 Riga	BT652	Gate opens	13:50
14:35 Inverness	BE7325	Gate opens	13:40
14:40 Salzburg	TCX386K	Gate opens	13:40
14:40 Anversa	ZB238	Gate opens	14:00
14:45 Aberdeen	BE7294	Gate opens	13:45
14:45 Sofia	MON6246	Gate opens	14:00
14:45 Newquay	BE804	Gate opens	14:05
14:50 Cork	FR9853	Gate opens	14:15
15:00 Shannon	FR1183	Gate opens	14:30
15:15 Guernsey	BE911	Gate opens	14:30
15:15 Marrakech	EZY5307	Gate opens	14:45
15:30 Guernsey	GR805	Gate opens	14:45
15:30 Girona	FR9229	Gate opens	14:30
15:30 Sofia	MON2274	Gate opens	15:10
15:55 Lisbon	EZY5455	Gate opens	15:20
16:05 Newcastle	BE466	Gate opens	15:20
16:10 Rome	EZY5257	Gate opens	15:20
16:15 Venice	EZY5265	Gate opens	15:20
16:25 Funchal	TP349	Gate opens	15:20
16:25 Belfast City	BE970	Gate opens	15:40
16:30 Geneva	BO7661	Gate opens	15:20

COLLAUDARE SISTEMI SOFTWARE...?

- Se il software fosse collaudato con la stessa cura e lo stesso rigore che si usano per automobili, ascensori, ponti...
si eviterebbero "figuracce" (e danni) come questi

..o come queste
a Roma Termini ☺



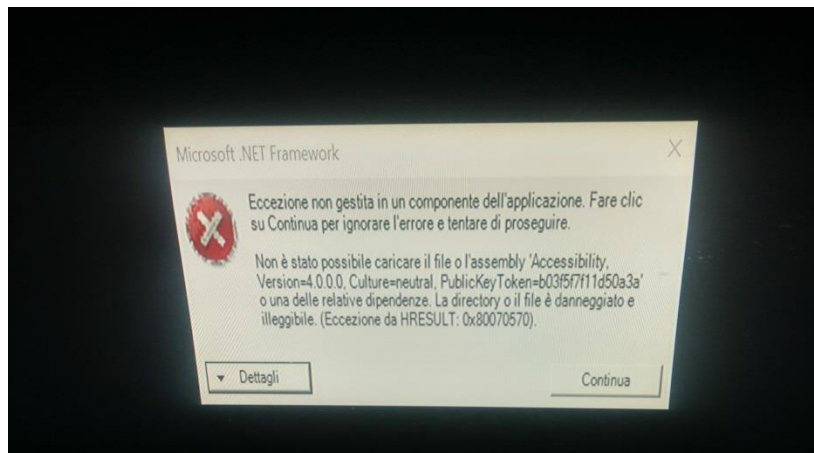
COLLAUDARE SISTEMI SOFTWARE...?

- Se il software fosse collaudato con la stessa cura e lo stesso rigore che si usano per automobili, ascensori, ponti...
si eviterebbero "figuracce" (e danni) come questi



COLLAUDARE SISTEMI SOFTWARE...?

- Se il software fosse collaudato con la stessa cura e lo stesso rigore che si usano per automobili, ascensori, ponti...
si eviterebbero "figuracce" (e danni) come questi



Ah, gli sportelli automatici...

COLLAUDARE SISTEMI SOFTWARE...?

- Se il software fosse collaudato con la stessa cura e lo stesso rigore che si usano per automobili, ascensori, ponti...
si eviterebbero "figuracce" (e danni) come questi

Per l'occasione, abbiamo riservato per te una speciale offerta:

Che dire? Un'offerta DAVVERO speciale... ☺

- cambio olio e filtro olio a 99â€ – la tua vettura e 129â€ – per il tuo v
- da oggi saremo aperti con orario continuato

Affrettati a cogliere questa opportunità!

Prezzi con o senza spazi?
Con la virgola o il punto?
Decidiamoci..

Su un contratto [redacted] è possibile [redacted] costo di 1,26 € al mese.
[redacted] che ti permette di ottenere sullo stesso contratto [redacted]
costo di 2,10€ al mese [redacted]

COLLAUDARE SISTEMI SOFTWARE...?

- Se il software fosse collaudato con la stessa cura e lo stesso rigore che si usano per automobili, ascensori, ponti...
si eviterebbero "figuracce" (e danni) come questi

Il famoso **PULSATE** bianco..

Mica male due errori nella stessa macchinetta.. *e sì che si vedono!* 😊

..e l'altrettanto famoso
CODICEE



COLLAUDO vs PROGETTO...?

- A volte invece sono le *assunzioni di base* non rispettate a generare assurdità al momento dell'uso..
...come in questa compagnia aerea europea

16 Posso prenotare un volo per un passeggero minorenne con il Codice di Prenotazione per due persone?

Se hai un Codice di Prenotazione per due persone uno dei due passeggeri può essere minorenne.

Il sito di prenotazione ti chiederà di inserire però la data di due passeggeri adulti. Per far sì che uno dei due passeggeri sia impostato come bambino, dovrai inserire una data fasulla di nascita e indicare la data di nascita corretta nel campo "Esigenze speciali di trasporto".

Stiamo lavorando per agevolare la prenotazione anche per i passeggeri minorenni, a breve sarà possibile procedere in automatico.

Per maggiori informazioni [CLICCA QUI](#).

Ma certo! Chi avrebbe mai potuto pensare che *anche un minorenne potesse prendere un aereo..?!?*

**Dominio del problema
mal analizzato**

Rimedio anche peggiore: MENTIRE nei moduli
e poi usare (malamente) un campo pensato per qualcos'altro!

COLLAUDO vs PROGETTO...?

- A volte invece sono le *assunzioni di base* non rispettate a generare assurdità al momento dell'uso..
...come in questo caso molto "bolognese" 😊

09:28	par	Bologna, ALDINI			Autobus 33
09:31	arr	Bologna, PORTA SAN MAMOLO			Bologna, PORTA SAN MAMOLO
09:39	par	Bologna, PORTA SAN MAMOLO			Autobus 33
09:45	arr	Bologna, CARDUCCI			Bologna, BOLOGNA STAZIONE CENTRALE

PIANIFICATORE 2016

Per andare da ALDINI a CARDUCCI basterebbe restare sul 33... e invece veniva suggerito di *scendere a porta S.Mamolo e aspettare il successivo!*

Che fosse per socializzare..? 😊

IN REALTÀ...

- chi ha scritto il software ha ipotizzato che tutte le linee abbiano due capilinea..
- *ma una linea circolare viola tale assunzione* ed è stata quindi modellata *come due tronconi*
"andata": Stazione/Porta S.Mamolo
"ritorno": Porta S.Mamolo/Stazione
- il software, tarato (giustamente) sull'assunzione di base, vede un itinerario composto.

COLLAUDO vs PROGETTO...?

- A volte invece sono le *assunzioni di base* non rispettate a generare assurdità al momento dell'uso..
...come in questo caso molto "bolognese" 😊

Partenza adesso
Opzioni

Invia indicazioni stradali al tuo telefono
Copia link

10:18—10:35	17 min
39	
10:14—10:32	18 min
11 > 🚶	
10:14—10:31	17 min
🚶 > 🚶 33	
10:24 da Porta San Mamolo	
🚶 12 min	ogni 12 min
Dettagli	



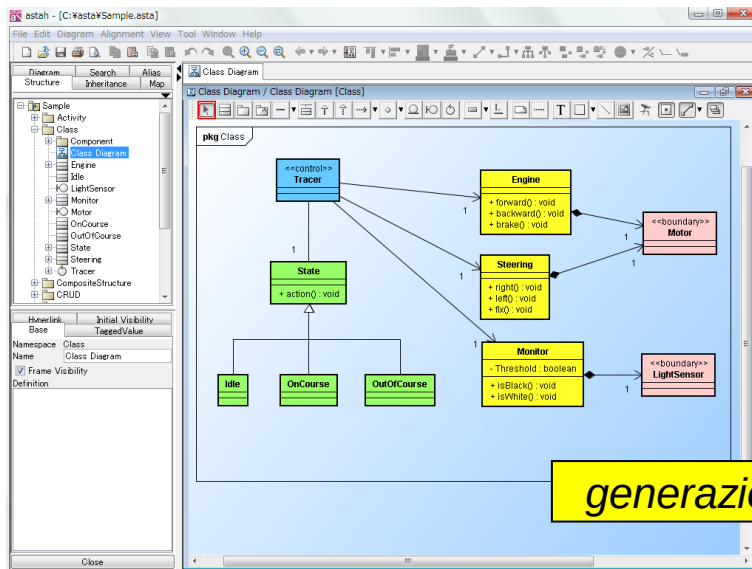


General Data Protection Regulation

- Dal 25/5/2018 ha sostituito la Data Protection Directive
- **Obbligo** di seguire i principi della GDPR
 - **privacy by design & by default**
(non solo misure tecniche, ma anche organizzative)
 - minimalità, proporzionalità
 - **anonimizzazione, pseudonimizzazione**
 - trasferimento dati fuori dalla EU (occhio al cloud!)
 - adeguatezza delle misure di sicurezza
- **Non è qualcosa che si possa "aggiungere dopo", a sistema già progettato: *va considerato fin dall'inizio***
 - ma non è (solo) un vincolo: è *un'opportunità per creare valore*

LA SFIDA DELL'AUTOMATIZZAZIONE

- Già da anni esistono strumenti che *generano lo scheletro del codice* partendo dal *modello del sistema*
- Il progettista deve solo aggiungere la *business logic*

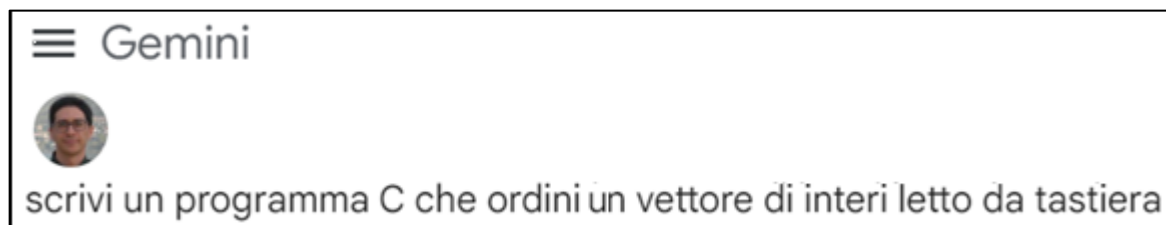


generazione codice

```
// Monitor.java
package Class;
public class Monitor {
    private boolean Threshold;
    private Tracer tracer;
    private LightSensor lightSensor;
    public void isBlack() {
        ...
    }
    public void isWhite() {
        ...
    }
}
```

LA SFIDA DELL'AUTOMATIZZAZIONE

- Oggi, strumenti di AI in rapida avanzata stanno diventando capaci di generare intere parti di codice



```
Inserisci la dimensione del vettore: 5
Inserisci gli elementi del vettore: 3
21
12
5
91
Vettore ordinato: 3 5 12 21 91
```

Programma C per ordinare un vettore di interi letto da tastiera:

```
C

#include <stdio.h>

void scambia(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

void ordinaBubbleSort(int *vettore, int n) {
    int i, j;
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (vettore[j] > vettore[j + 1]) {
                scambia(&vettore[j], &vettore[j + 1]);
            }
        }
    }
}
```

```
int main() {
    int n, i;

    // Inserimento dimensione del vettore
    printf("Inserisci la dimensione del vettore: ");
    scanf("%d", &n);

    // Dichiarazione del vettore
    int vettore[n];

    // Lettura elementi del vettore
    printf("Inserisci gli elementi del vettore: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &vettore[i]);
    }

    // Ordinamento del vettore
    ordinaBubbleSort(vettore, n);
}
```

```
// Stampa del vettore ordinato
printf("Vettore ordinato: ");
for (i = 0; i < n; i++) {
    printf("%d ", vettore[i]);
}

printf("\n");

return 0;
}
```



LA SFIDA DELL'AUTOMATIZZAZIONE

- Senz'altro utile per alleggerirsi dai compiti ripetitivi
 - MA occhio al risultato: i generatori di testo sono progettati più per produrre *testo gradevole* che *testo preciso*
 - serve sempre una mente lucida e *competente* per valutare ciò che viene prodotto!
- In prospettiva, si liberano tempo ed energie per concentrarsi ancor più sul *progetto*, la *vera dimensione dell'Ingegneria*
 - saremo sempre più *direttori d'orchestra*, non «programmatori»
 - servirà sempre *maggior competenza*, non meno

IL VOSTRO FUTURO...

- Saper progettare **sistemi di qualità** è e sarà sempre più necessario *per la vostra professionalità e il vostro futuro*

Assunzioni non stagionali fino a 29 anni pre Prime 32 figure professionali ⁽¹⁾ p				24 Dicembre 2012	IL SOLE 24 ORE	estratto
Professione	Cat. ⁽²⁾	Valore assoluto 2012	Ir	Il bilancio		
Commessi delle vendite al minuto	PCS	23.520		Le caratteristiche delle assunzioni under 30 programmate dalle imprese per il 2012		
Addetti alla gestione dei magazzini e professioni assimilate	I	5.740		200mila Le assunzioni di under 30 Sono gli inserimenti di giovani programmati dalle imprese per il 2012, in flessione di circa il 30% rispetto al 2011	33% Non stagionali Più di un'assunzione su tre non stagionale (circa 144mila) è rivolta ad under 30, ma il numero si è dimezzato rispetto al periodo pre-crisi	40% Primato del Centro Nel Centro Italia 4 assunzioni su dieci sono per giovani; nel Settentrione si registrano, invece, le dinamiche peggiori rispetto al 2011 (-32%)
Camerieri e professioni assimilate	PCS	5.560				
Addetti agli affari generali	I	5.270		13mila Milano e Roma al top Nelle province di Milano e Roma si registrano più assunzioni in valore assoluto (entrambe oltre quota 13mila); all'opposto Ragusa, Prato e Pisa	24mila Il record dei commessi I profili più richiesti sono i commessi, seguiti da magazzinieri, camerieri, impiegati e segretari (tutti con oltre 5mila assunzioni)	2.900 Progettisti software Tra le figure «high skilled», si distinguono progettisti software (2.900), contabili (2.500) e tecnici per la vendita e la distribuzione (2.200)
Addetti a funzioni di segreteria	I	5.150				
Personale non qualif. servizi pulizia uffici ed esercizi commerciali	PNQ	3.410				
Baristi e professioni assimilate	PCS	3.270				
Cuochi in alberghi e ristoranti	PCS	3.130				
Analisti e progettisti di software	PIS	2.890				
Addetti alla preparazione, alla cottura e alla distribuzione di cibi	PCS	2.860				
Muratori in pietra, mattoni, refrattari	OS	2.620				
Contabili e professioni assimilate	PT	2.530				
Facchini, addetti allo spostamento merci	PNQ	2.500				
Addetti sportelli assicurativi, bancari e altri intermed. finanziari	I	2.350				
Tecnici della vendita e della distribuzione	PT	2.240				



Lavoro: ecco le figure più ricercate ma che le aziende non trovano. Situazione nelle varie Regioni

Ha molte possibilità di sbocco anche l'addetto all' (tra i laureati) con 2.950 assunzioni previste e il 23 reperimento. Molto facile per le aziende invece tro

Considerando invece le professioni di maggiore sbocco occupazionale, sempre tra i "dottori",

GLI INTROVABILI (2013)

Commissione Europea & Il Sole24ore



About this Site | Contact | FAQ | Cookies | Search | Legal notice

European Commission

EMPLOYMENT, SOCIAL AFFAIRS & INCLUSION

European Commission > Employment, Social Affairs & Inclusion > ... > News

POLICIES AND ACTIVITIES | NEWS AND MULTIMEDIA | PUBLICATIONS AND DOCUMENTS

Search

The top 5 occupations with best employment opportunities were:

1. Personal care workers in health services
2. Software applications developers and analysts
3. Administrative and specialised secretaries
4. Mining, manufacturing and construction supervisors
5. Primary school and early childhood teachers

GRAFICI

Assunzioni: consistenza e andamento



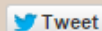
Le prime cinque professioni tradizionali in crescita
L'Osservatorio europeo dei posti di lavoro vacanti (Evm) segnala cinque occupazioni tradizionali tra le più richieste e in crescita di assunzioni: servizio alla persona nell'ambito della sanità sviluppatore di applicazioni software e analista, segretario amministrativo e specializzato, supervisore di miniera, di produzione e di costruzione e insegnante elementare e per la prima infanzia. L'agenda per nuove competenze e nuove occupazioni nell'ambito di Europa 2020 punta il faro sul numero insufficiente di infermieri formati e un numero crescente di infermieri e ostetrici vicini all'età pensionabile. Oltre ai tecnici medici e farmaceutici sono professioni in espansione nell'ambito sanitario anche dentisti, farmacisti e fisioterapisti.

GLI INTROVABILI (2014)

Indagine Unioncamere (*Il Sole24ore*, 21/11/2014)

Dall'esperto software allo stuccatore: introvabile una professione su 10

di [Alessia Tripodi](#) 21 novembre 2014 Commenti (15)



Tweet 15



Consiglia

213



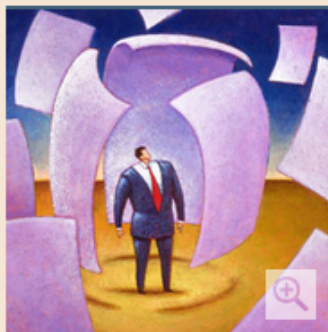
g+ 4



My24



A A



Ingegneri in Lombardia, stuccatori edili e musicisti in Veneto, accompagnatori turistici nel Lazio, organizzatori di eventi in Piemonte. È questa la mappa delle figure "introvabili" per le imprese italiane secondo i dati dell'Osservatorio Excelsior di Unioncamere presentati ieri a Verona in occasione di «Job&Orienta».

DOCUMENTI



▼ [I numeri del gap](#)

Secondo i dati, nel 2014 le lauree più ricercate ma più difficili da reperire sono quelle in ingegneria, soprattutto in Lombardia. Delle 530 assunzioni di ingegneri esperti di software in tutta Italia, infatti, 210 (circa il 40%) sono considerate introvabili, così come i 300 profili di analista programmatore sui 930 ricercati in tutto il Paese nel 2014 (32%). Nella classifica delle

GLI INTROVABILI (2015)

Unioncamere (*Il Sole24ore*, 27/11/2015)

In Italia ci sono 76mila profili professionali introvabili

Statistici, ingegneri, analisti e progettisti, ma anche tanti operatori nel settore del mobile e arredamento, abbigliamento, moda ed elettronico

Adriano Palazzolo - Ven, 27/11/2015 - 15:50



Se gli statistici, **gli ingegneri** e i laureati in materie scientifiche "sono i più difficili da reperire" tra i dottori, problemi non trascurabili sono evidenziati anche tra i diplomati e quanti hanno conseguito una qualifica professionale. Sebbene per questi ultimi le difficoltà di reperimento siano mediamente inferiori a quella dei laureati (rispettivamente, il 10 e il 9,1% del totale delle assunzioni), per alcuni indirizzi le imprese segnalano quote anche consistenti di "introvabili".

Tra i diplomati, le difficoltà salgono decisamente sopra la media nel caso dell'indirizzo **telecomunicazioni** (20,4%, pari a 150 delle 760 unità programmate), dell'indirizzo informatico (19,1%, per 970 unità delle 5.060) e dell'indirizzo grafico-pubblicitario (18,8%, per 230 delle 1.230 assunzioni previste).

Tra i candidati con qualifica di formazione professionale, i problemi nel reperimento sono particolarmente sensibili nel caso dell'indirizzo **tessile**, abbigliamento e moda, arrivando a quasi un'assunzione su tre, mentre si attestano intorno a una su 6 nel caso dell'indirizzo legno-mobile-arredo e di quello elettrotecnico.

Riguardo le professioni dei laureati, Unioncamere registra oltre 2mila opportunità per gli analisti informatici (2.110 i ricercati, **la metà dei quali introvabili**), seguiti dai **progettisti** di impianti per l'automazione industriale (670 totali, 210 di difficile reperimento), quindi i tecnici specialisti di applicazioni informatiche (3.130, 540 dei quali ritenuti difficili da reperire).

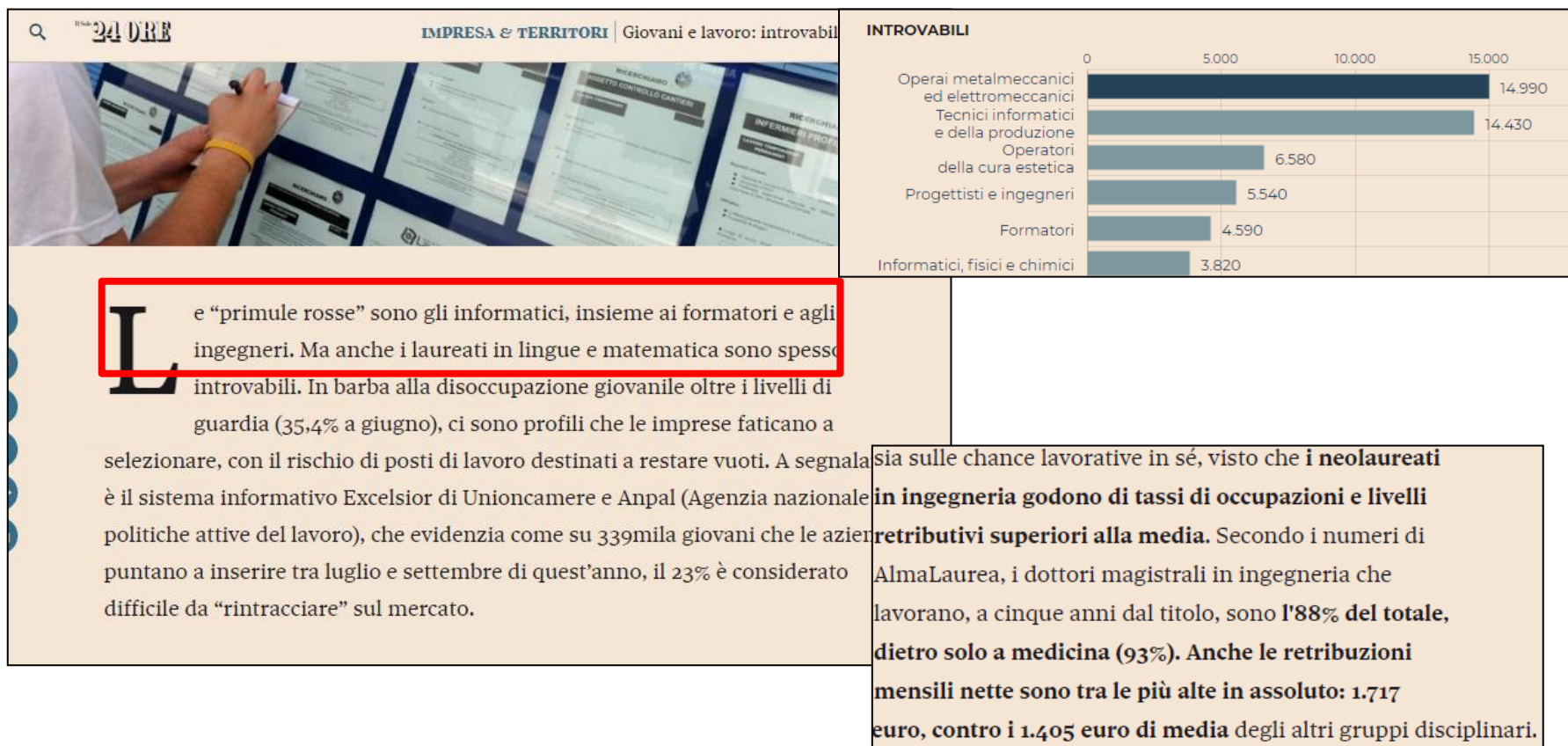
(*Panorama*, 16/2/2015)

I dati

Secondo le ricerche condotte dall'agenzia di selezione del personale Page Personnel, i profili professionali con 3 - 5 anni di esperienza più richiesti nel nostro Paese trovano sbocco innanzitutto nell'area trasporti e spedizioni, che si rivela un mercato in continua espansione. Ottime prospettive anche per tecnici e ingegneri, con un focus su **ingegneri informatici e ingegneri meccanici** che possono aspirare a professioni ben retribuite, ben tutelate e che assicurano esperienze di lavoro stimolanti ed interessanti, ma ancora molto di nicchia e poco conosciute. Infine, il settore digital si fa notare perché è caratterizzato da una **disoccupazione prossima allo zero**.

GLI INTROVABILI (2017)

Unioncamere (*Il Sole24ore*, 2017)



GLI INTROVABILI (2018)

Unioncamere (2018)



Most sought-after jobs in Germany

The ten most frequently sought jobs in Germany:

1 Software developer and programmer

2 Electronics engineer, electrician

3 Healthcare worker and nurse

4 IT consultant, IT analyst

5 Economist, business administrator

6 Account manager, client consultant

7 Production assistant

8 Sales representative, sales assistant

9 Sales manager, product manager

10 Architect, structural engineer

Source: DEKRA Akademien 2018

GLI INTROVABILI (2020)

Unioncamere (2020)

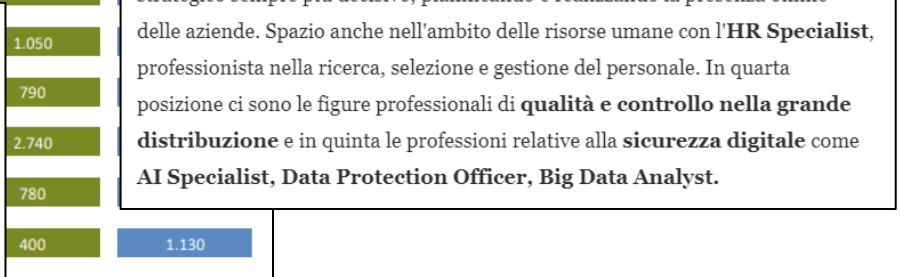
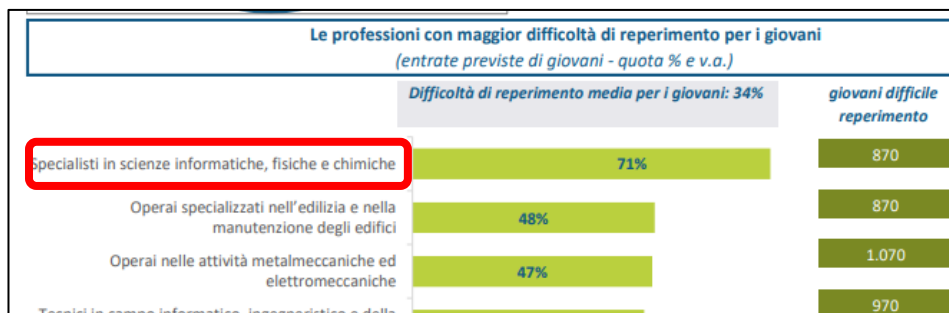
Le 5 professioni più richieste, secondo Union Camere

L'analisi del Sistema Informativo Excelsior promosso da Union Camere e Anpal evidenzia che le professioni più richieste sono cinque e sono ben distribuite in tutti i settori. La prima riguarda il settore della programmazione e sviluppo è in assoluto l'ambito lavorativo più dinamico degli ultimi anni con il digitale che ha acquisito un ruolo strategico essenziale in tutte le aziende. Il **Developer**, l'**ingegnere informatico** che si occupa di progettare, sviluppare e gestire non solo siti internet e app, ma anche codici per sistemi operativi e software di uso interno all'azienda, andando a supportare così il percorso di Digital Transformation che tutte le imprese stanno affrontando.

In seconda posizione c'è il **Web Marketing Specialist** che assume un ruolo strategico sempre più decisivo, pianificando e realizzando la presenza online delle aziende. Spazio anche nell'ambito delle risorse umane con l'**HR Specialist**, professionista nella ricerca, selezione e gestione del personale. In quarta posizione ci sono le figure professionali di **qualità e controllo nella grande distribuzione** e in quinta le professioni relative alla **sicurezza digitale** come **AI Specialist**, **Data Protection Officer**, **Big Data Analyst**.

Le professioni più richieste nell'ambito tech

La ricerca di **Experis** invece si è focalizzata sulle figure tech più ricercate del 2021. Si tratta di professionisti diventati fondamentali per la transizione tecnologiche delle imprese, e che possono percepire stipendi molto generosi, proprio per la loro specializzazione. Sono i **developer**, **machine learning specialist**, **cloud architect**, **cyber security manager**, **data scientist/data analyst**, e **system administrator**. Se vi state chiedendo cosa siano e in cosa consistano, sappiate che hanno a che fare con la crescita impetuosa dell'**intelligenza artificiale** e del **machine learning**, i nuovi metodi di programmazione che permettono a computer e robot di apprendere informazioni partendo dai dati. I comparti che si stanno dotando maggiormente di questi profili sono quelli dei **servizi**, **finanza**, **telecomunicazioni**, **pharma** e **sanità**.



Gli indirizzi di studio più richiesti

	Entrate previste (v.a.)	% sul totale	% diff. reperimento	% con esperienza
	31.510	16%	40%	84%
Indirizzo ingegneria	8.520		45%	80%
Indirizzo economico	8.020		31%	78%
Insegnamento e formazione	3.840		40%	91%
Indirizzo sanitario e paramedico	2.930		44%	93%
Indirizzo scienze matematiche, fisiche e informatiche	1.820		56%	91%

GLI INTROVABILI (2021)

Unioncamere (2021-22)

I profili STEM maggiormente difficili da reperire

Q: Di quali figure in ambito STEM in particolare avrebbe bisogno la sua azienda?



INGEGNERI
MECCANICI



INGEGNERI
AUTOMAZIONE



INGEGNERI
INFORMAZIONE

Imprese che hanno cercato risorse STEM ma non le hanno trovate
tutte, in collaborazione con SWG e grazie al contributo di Monitor Deloitte

Gli indirizzi di studio più richiesti

	Entrate previste (v.a.)	% sul totale	% diff. reperimento	% con esperienza
UNIVERSITARIO	88.330	19%	43%	87%
Indirizzo economico	25.360		27%	82%
Indirizzo ingegneria	21.670		53%	90%
Indirizzo insegnamento e formazione	9.580		42%	92%
Indirizzo sanitario e paramedico	6.960		60%	91%
Indirizzo scienze matematiche, fisiche e informatiche	6.560		70%	86%

Le professioni con maggior difficoltà di reperimento per i giovani

(entrate previste di giovani - quota % e v.a.)

Difficoltà di reperimento media per i giovani: 40%

		giovani difficile reperimento	entrate giovani
Specialisti in scienze informatiche, fisiche e chimiche	68	1.030	1.520
Operai specializzati nell'edilizia e nella manutenzione degli edifici	63	3.020	4.780
Conduuttori di mezzi di trasporto	57	1.750	3.090
Tecnici della sanità, dei servizi sociali e dell'istruzione	56	1.710	3.070

GLI INTROVABILI (2022)

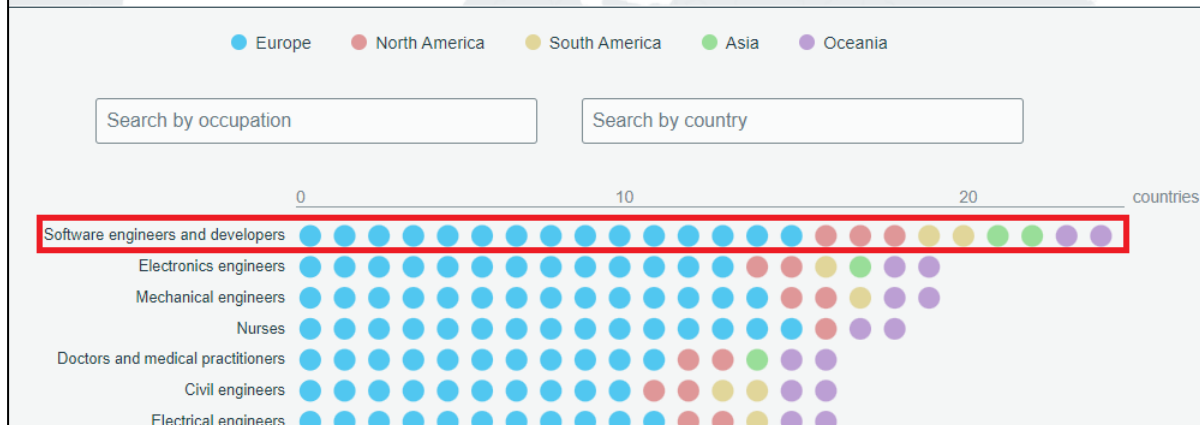
LE PROFESSIONI PIÙ RICHIESTE NEL MESE DI GENNAIO 2022 *

secondo la classificazione delle professioni - ISTAT 2011 (3 digit)

	ENTRATE PREVISTE GENNAIO 2022	Ripartiz x 1000	Difficoltà a reperire (%)	ENTRATE PREVISTE gennaio - marzo 2022
ENTRATE	457.650	1.000	38,6	1.158.150
DIRIGENTI	1.340	2,9	54,0	2.510
PROFESSIONI INTELLETTUALI, SCIENTIFICHE E DI ELEVATA SPECIALIZZAZIONE	35.580	77,7	43,2	77.350
Specialisti in scienze matematiche, informatiche, chimiche, fisiche e naturali	5.920	12,9	61,9	13.470
Ingegneri e professioni assimilate	7.400	16,2	52,9	15.280
Specialisti delle scienze gestionali, commerciali e bancarie	7.490	16,4	33,8	14.550
Specialisti in discipline artistico-espressive	2.100	4,6	17,0	4.820
Altri specialisti dell'educazione e della formazione	2.500	5,5	36,9	5.150

THE WORLD'S MOST IN DEMAND PROFESSIONS

Across the developed countries of the world, skilled professionals are in high demand. Software engineers are needed in 24 countries, nurses are needed in 18, while 11 countries report a shortage of accountants. Explore the chart below to see which occupations are most in demand across the world and discover which skills are needed.

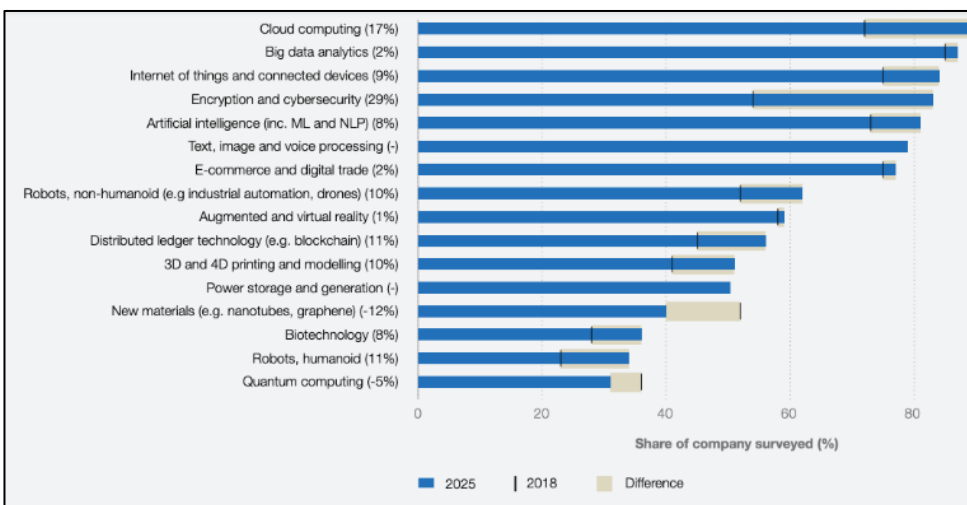


GLI INTROVABILI (2023)

LE PROFESSIONI PIÙ RICHIESTE NEL MESE DI GENNAIO 2023 *

secondo la classificazione delle professioni - ISTAT 2011 (3 digit)

	ENTRATE PREVISTE GENNAIO 2023	Ripartiz x 1000	Difficoltà a reperire (v.a.)	Difficoltà a reperire (%)	ENTRATE PREVISTE gennaio - marzo 2023
ENTRATE	503.670	1.000	229.455	45,6	1.307.070
DIRIGENTI	1.840	3,6	1.216	66,1	4.040
PROFESSIONI INTELLETTUALI, SCIENTIFICHE E DI ELEVATA SPECIALIZZAZIONE	40.540	80,5	19.255	47,5	93.560
Specialisti in scienze matematiche, informatiche, chimiche, fisiche e naturali	7.550	15,0	3.667	48,6	18.730
Ingegneri e professioni assimilate	7.890	15,7	4.179	53,0	18.430
Specialisti nelle scienze della vita	3.750	7,4	2.632	70,2	7.610
Specialisti delle scienze gestionali, commerciali e bancarie	7.330	14,5	2.736	37,3	16.730
Altri specialisti dell'educazione e della formazione	3.250	6,5	1.541	47,4	6.480
PROFESSIONI TECNICHE	86.020	170,8	44.415	51,6	201.410
Tecnici informatici, telematici e delle telecomunicazioni	10.940	21,7	6.361	58,1	26.560
Tecnici in campo ingegneristico	8.690	17,2	5.192	59,7	22.250
Tecnici della gestione dei processi produttivi di beni e servizi	6.510	12,9	4.161	63,9	16.610
Tecnici della salute	14.830	29,4	9.311	62,8	31.380
Tecnici dell'organizzazione e dell'amministrazione delle attività produttive	8.430	16,7	3.579	42,5	17.930
Tecnici delle attività finanziarie e assicurative	2.960	5,9	1.210	40,9	6.230



Gli indirizzi di studio più richiesti

	Entrate previste (v.a.)	% sul totale
	96.060	19%
Indirizzo economico	25.940	
Indirizzo ingegneria	23.300	
amento e formazione	11.500	
anitario e paramedico	9.220	
Indirizzo scienze matematiche, fisiche e informatiche	5.710	

TOP FIVE IN-DEMAND ROLES

Whoever holds the talent holds the future. These are the most sought-after professions by employers globally.

- 1 IT & DATA**
- 2 SALES & MARKETING**
- 3 OPERATIONS & LOGISTICS**
- 4 MANUFACTURING & PRODUCTION**
- 5 CUSTOMER FACING & FRONT OFFICE**



DIVENTARE INGEGNERI INFORMATICI

- Per diventare validi ingegneri informatici bisogna "solo" *saper programmare bene*, in linguaggi moderni e di moda?

NO !

- Serve molto di più:
 - *saper pensare*, riflettere sulle cose (anche filosoficamente)
 - *saper impostare l'attività di progetto* di un sistema complesso
 - comprendere che il progetto non è il codice
 - comprendere che progettare non è implementare
 - interiorizzare l'idea che il collaudo è parte integrante della propria attività, non qualcosa "da fare dopo" o delegabile ad altri



IL PROBLEMA DEL PROGETTO

Problema:

spesso si pensa (solo) in base ai linguaggi conosciuti

MA

i linguaggi "classici" (C, ...) non offrono strumenti mentali tarati per affrontare la progettazione di sistemi software complessi

Problema del "foglio bianco":

- da dove si parte?
- come si comincia a "immaginare" la soluzione ?
- le metodologie, gli approcci usati per affrontare problemi "piccoli" (algoritmi) scalano su problemi "grandi" (sistemi software)?
Si riescono ancora a usare con efficacia?



IL TEMA DELLA SCALABILITÀ

- L'essere umano ha la naturale tendenza a pensare che tutto sia *scalabile*
 - ha funzionato bene su 3 elementi, perché non dovrebbe funzionare altrettanto bene con 300 o 3000 o 300000..?
 - ho sempre usato questo approccio per progettare finora, perché non dovrebbe reggere anche più in grande domani?
 - ho sempre fatto così, perché mi vuoi costringere a cambiare?
- Purtroppo, ciò è spesso una *illusione mentale*
 - l'essere umano è *intrinsecamente pigro*: ripensare ai propri approcci, cambiare abitudini e metodi di lavoro *costa fatica*
 - ..ma nei fatti *gli strumenti (fisici e mentali) quasi mai scalano all'aumentare della complessità!*
 - per avvitare cinque viti basta un cacciavite.. ma per la Tour Eiffel?



LINGUAGGI & CONTESTO

I linguaggi "classici", come il C:

- sono nati quando l'informatica era allo *stadio artigianale*
 - l'attenzione era sul definire un "linguaggio per codificare algoritmi"
 - i programmi erano piccoli sistemi, gestiti tipicamente da una singola persona (o poche persone) secondo il proprio stile di lavoro
 - l'hardware non era certo quello di oggi..!
- *non erano pensati per supportare un'attività di progetto*
 - il progetto "era" il codice (o al massimo un diagramma di flusso..)
- *non erano pensati per supportare una attività di collaudo sistematica di carattere ingegneristico - industriale*
 - il collaudo (quando c'era..) era un'attività "*fai da te*", non strumentata
 - i test (quando esistevano) erano tipicamente *mischiati nel codice*



LINGUAGGI & CONTESTO

Un tale approccio oggi **non può reggere «nel grande»:**

- è *impensabile* progettare e mantenere col "fai da te" un sistema complesso, costoso e spesso "mission critical"
- **il linguaggio diviene solo *uno* degli strumenti di una *attività ben più ampia di progetto e sviluppo*, che va organizzata**
- soprattutto, *il linguaggio non può più limitarsi a fornire solo gli elementi linguistici*, lasciando al progettista l'onere di affrontare tutta la complessità:
deve fornire concetti, metafore e strumenti adatti a governare la complessità dei sistemi software



VERSO NUOVI LINGUAGGI ED ECOSISTEMI

I modelli di progettazione e i linguaggi moderni sono nati per cercare di superare questo limite

- al di là delle differenze, essi introducono *nuovi concetti, nuove metafore e nuovi strumenti mentali* per riuscire a *concepire e governare il progetto, lo sviluppo e il collaudo* di sistemi software complessi
- la varietà di metafore e concetti è *la ragione di fondo per l'esistenza di tanti linguaggi* e il loro continuo proliferare

I modelli e linguaggi A OGGETTI costituiscono uno dei *maggiori casi di successo* degli ultimi decenni

ultimamente sempre più spesso **BLENDED** con concetti chiave provenienti dai **LINGUAGGI FUNZIONALI**



NEW LIFE TO OLD IDEAS

Paradigmi funzionali alla riscossa

- Per decenni, i paradigmi di programmazione e i linguaggi *funzionali*, nonostante il notevole contributo di idee "forti", sono stati penalizzati dalla *sintassi poco user-friendly*, *inadatta al "grande pubblico"*
 - spesso confinati nell'accademia, o in gruppi di "affezionati utenti"
 - spesso visti "con sospetto" dai "veri programmatori" (...)
- Ma nell'ultimo decennio *l'aumento della complessità dei sistemi* ha dato loro nuova linfa in nuove, moderne forme
 - *LINGUAGGI BLENDED: idee originate dal paradigma funzionale re-innestate e adattate sui moderni linguaggi "main stream"* – sfruttando le infrastrutture esistenti 😊



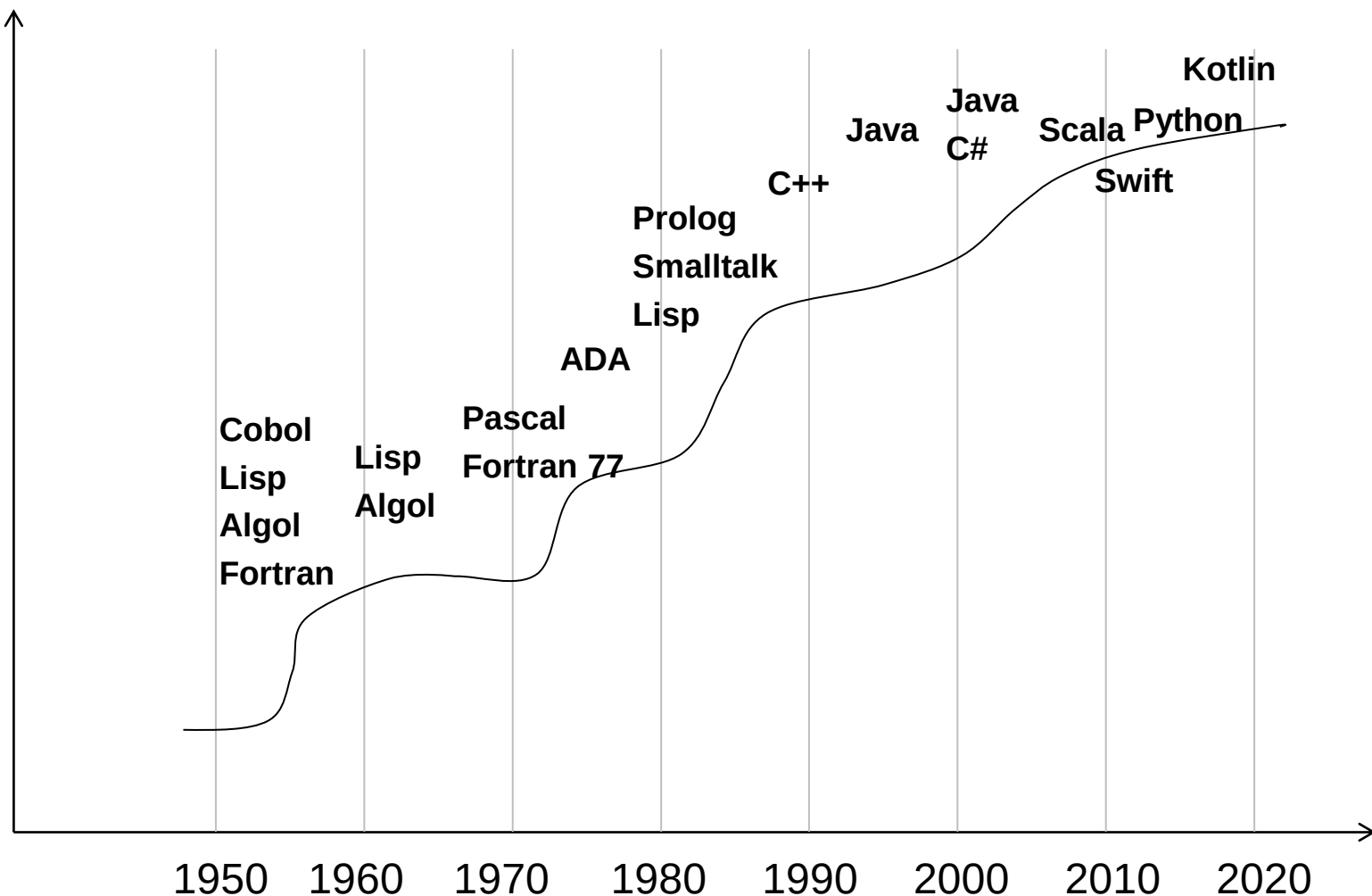
NUOVI PUNTI DI VISTA, NUOVE ISPIRAZIONI

Alcune idee "from the functional programming world"

- distinzione variabili / valori (**var** vs **val**)
- abolizione dei tipi primitivi: *"everything is an object"*
- costrutti come espressioni: *"everything is an expression"*
- strutture dati imm modificabili: *"compute by synthesis"*
- funzioni come first-class entities
 - *chiusure, lambda expression*
 - *operatori come funzioni, operatori come metodi*
 - *lazy evaluation*
- stile più conciso
- ...



VERSO NUOVI LINGUAGGI





VERSO NUOVI LINGUAGGI

Scala

Scala 1.0	2004
Scala 2.0	2006
...	
Scala 2.13	2019
...	
Scala 3.3.1	2023

Kotlin

kotlin 1.0	2016
kotlin 1.2	2017
kotlin 1.3	2018
Kotlin 1.6	2021
...	
Kotlin 1.9	2023

C# history

C# 1.0	2002
C# 2.0	2005
C# 3.0	2007
C# 4.0	2010
C# 5.0	2012
C# 6.0	2015
C# 7.0	2017
C# 8.0	2019
C# 9.0	2020
C# 10.0	2021
C# 11.0	2022
C# 12.0	2023

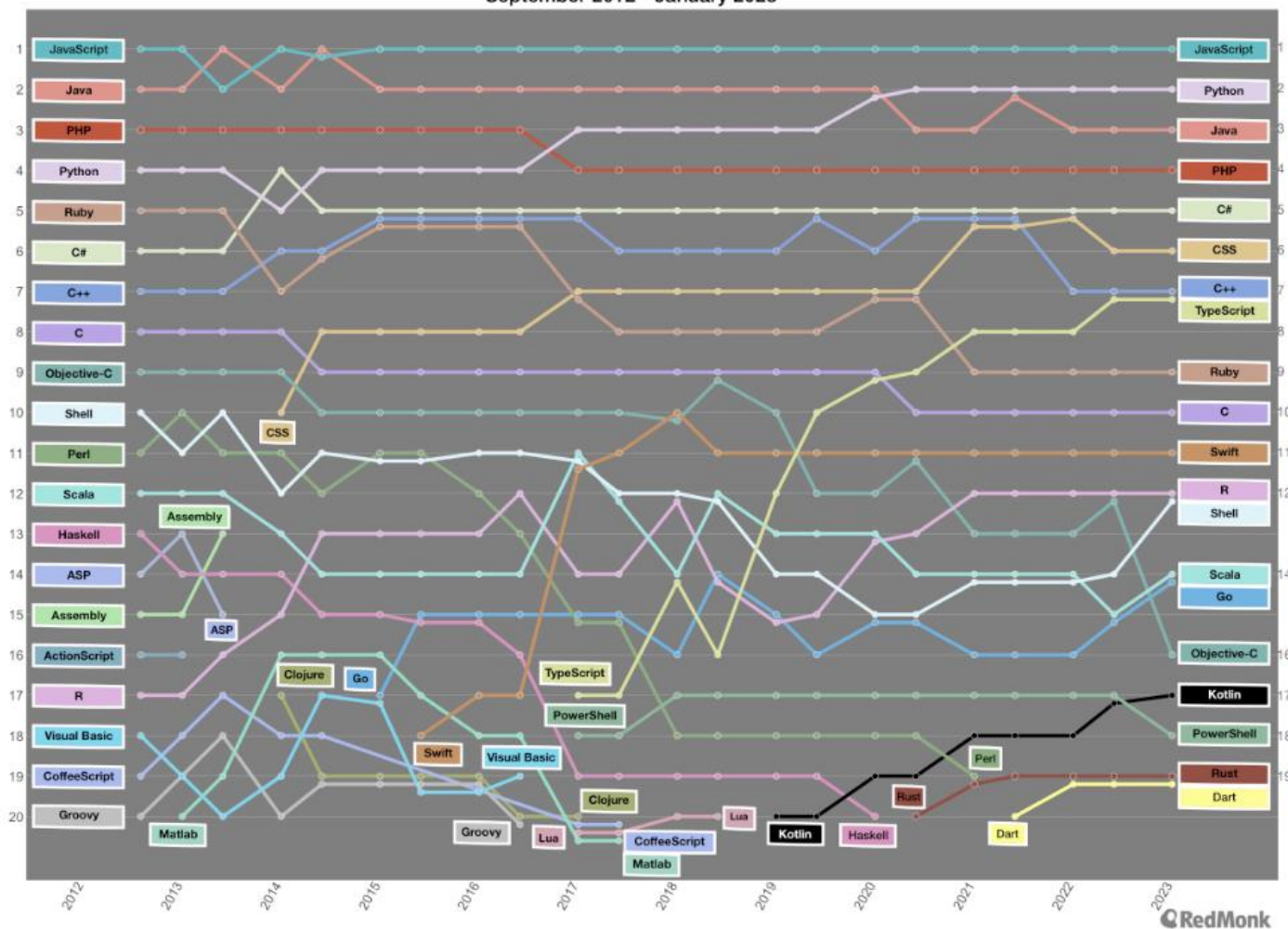
Java history

JDK 1.0 (1996)
JDK 1.1 (1997)
J2SE 1.2 (1998)
J2SE 1.3 (2000)
J2SE 1.4 (2002)
J2SE 5.0 (2004)
Java SE 6 (2006)
Java SE 7 (2011)
Java SE 8 (2014)[LTS]
Java SE 9 (2017)
Java SE 11 (2018)[LTS]
...
Java SE 17 (2021) [LTS]
Java SE 21 (2023) [LTS]

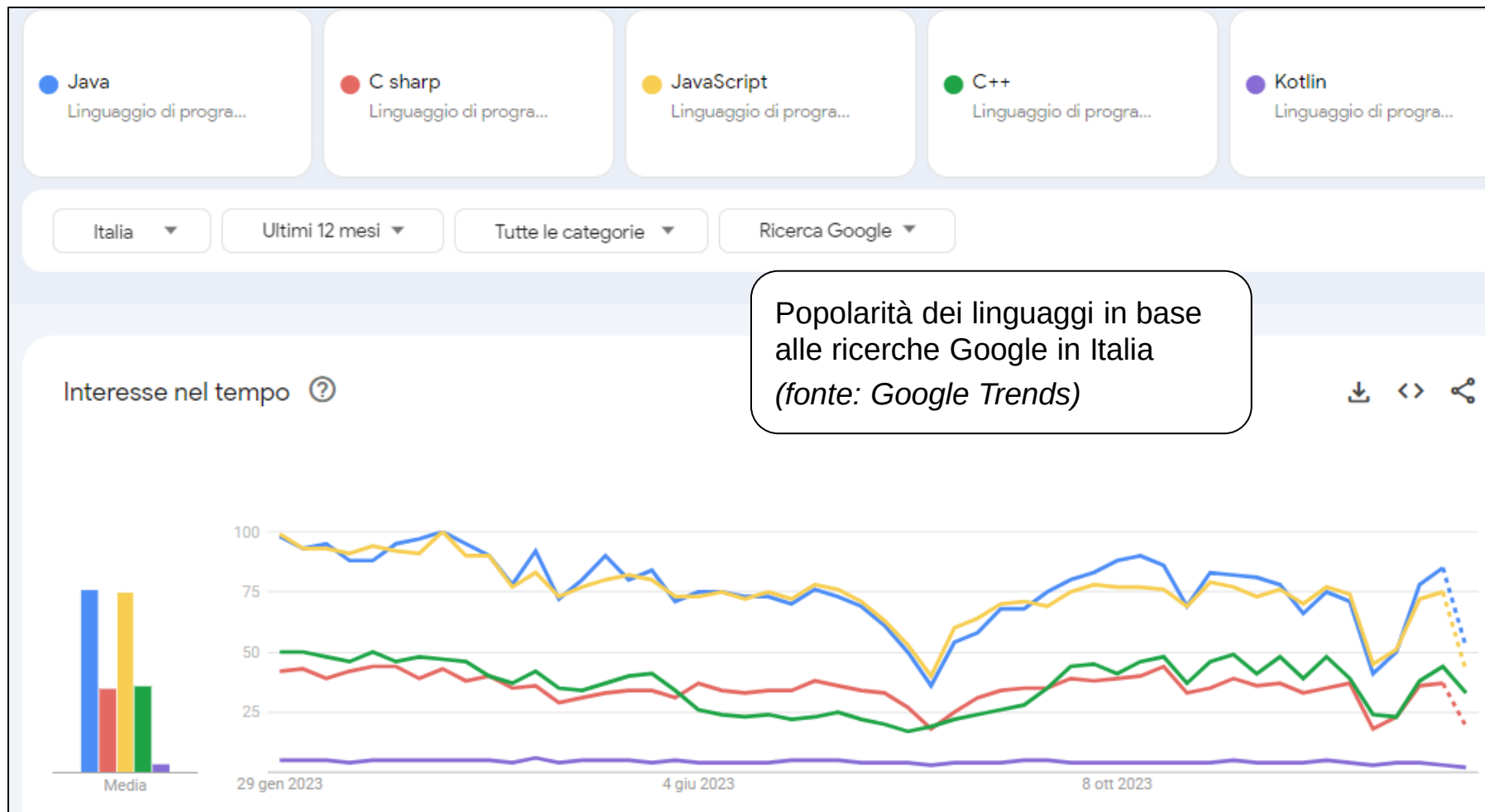
VERSO NUOVI LINGUAGGI

RedMonk Language Rankings

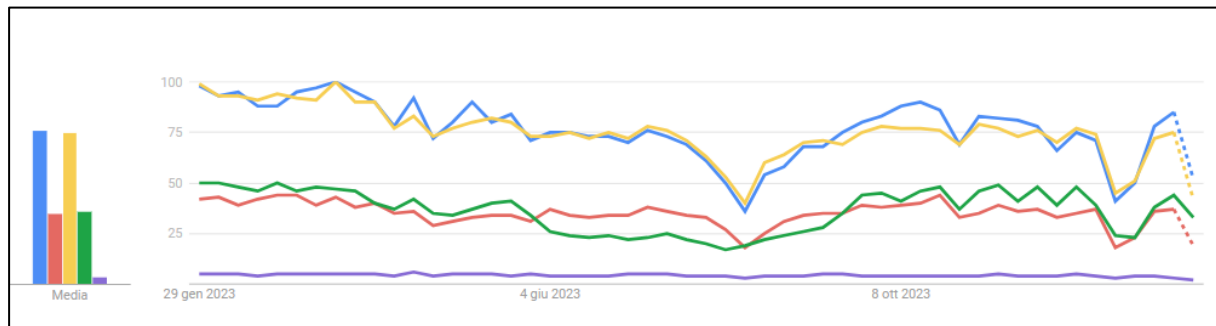
September 2012 - January 2023



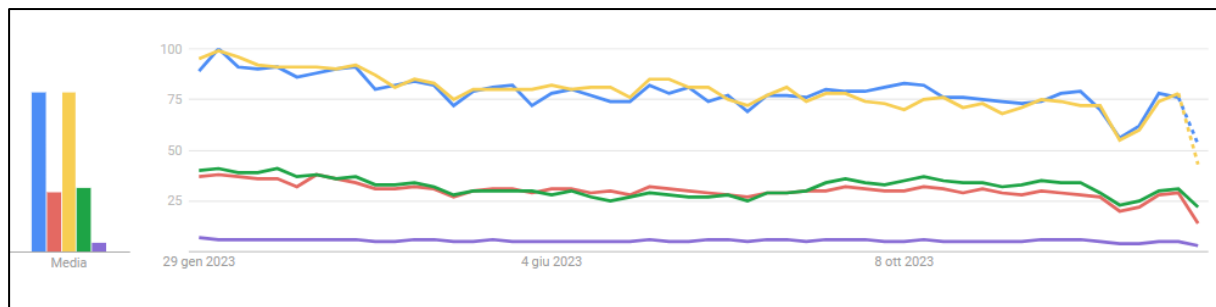
VERSO NUOVI LINGUAGGI



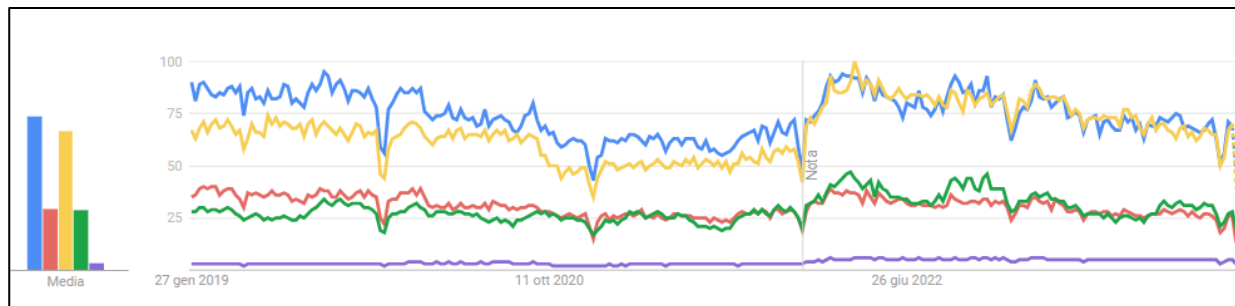
VERSO NUOVI LINGUAGGI



Italia
1 anno



Mondo
1 anno



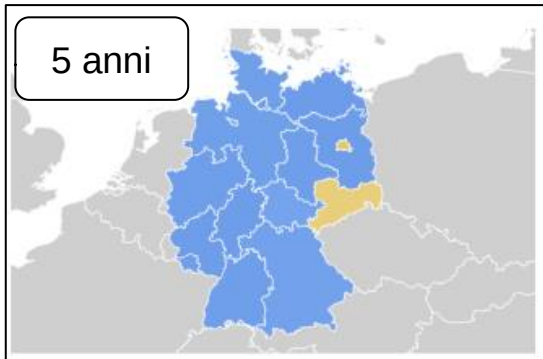
Mondo
5 anni

VERSO NUOVI LINGUAGGI

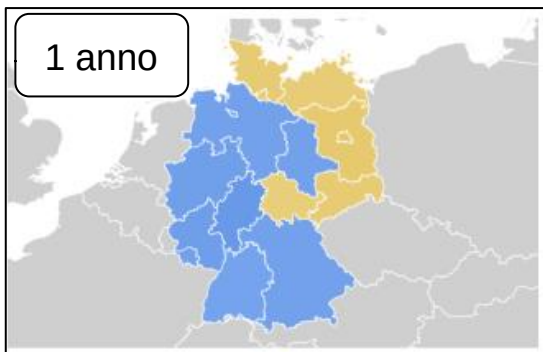
Popolarità dei linguaggi in base
alle ricerche Google

(fonte: Google Trends)

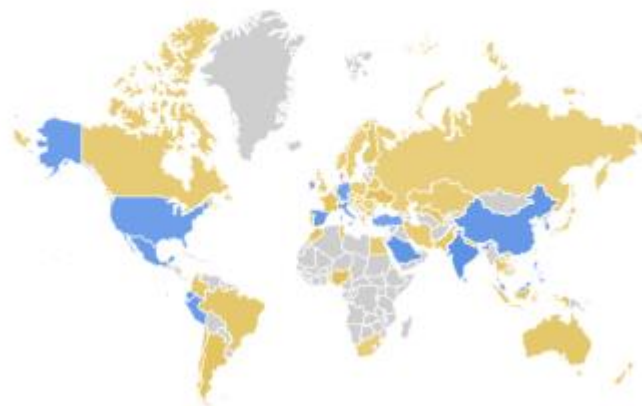
5 anni



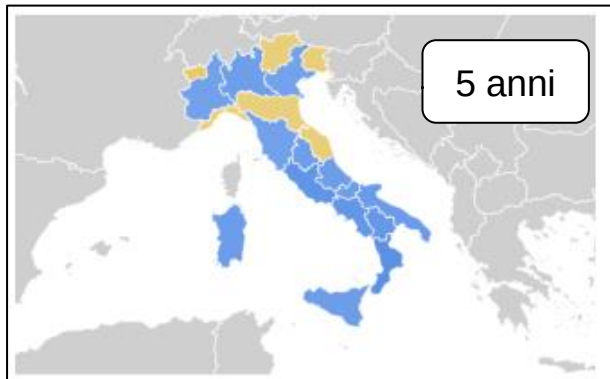
1 anno



● Java ● C sharp ● JavaScript ● C++ ● Kotlin



5 anni



1 anno



VERSO NUOVI LINGUAGGI

PYPL Popularity of Programming Language



Jan
2024



Jan
2023

Worldwide, Jan 2024 :				
Rank	Change	Language	Share	1-year trend
1		Python	28.2 %	+0.5 %
2		Java	15.73 %	-0.9 %
3		JavaScript	8.91 %	-0.6 %
4	↑	C/C++	6.8 %	-0.0 %
5	↓	C#	6.67 %	-0.3 %
6	↑	R	4.59 %	+0.6 %
7	↓	PHP	4.54 %	-0.7 %
8		TypeScript	2.92 %	+0.2 %
9		Swift	2.77 %	+0.6 %
10		Objective-C	2.34 %	+0.2 %
11	↑	Rust	2.19 %	+0.3 %
12	↓	Go	2.02 %	+0.1 %
13		Kotlin	1.78 %	-0.0 %
14		Matlab	1.59 %	-0.0 %
15		Ruby	1.03 %	+0.0 %
16	↑↑	Ada	1.02 %	+0.2 %
17	↑↑	Dart	0.99 %	+0.2 %
18	↓↓	VBA	0.91 %	-0.1 %
19	↓↓	Powershell	0.91 %	+0.0 %
20	↑↑	Lua	0.71 %	+0.1 %
21	↓	Scala	0.61 %	-0.0 %



VERSO NUOVI LINGUAGGI

Jan 2017

Worldwide, Jan 2017 compared to a year ago:				
Rank	Change	Language	Share	Trend
1		Java	23.1 %	-0.9 %
2		Python	14.4 %	+2.7 %
3		PHP	9.7 %	-1.0 %
4		C#	8.4 %	-0.1 %
5	↑↑	Javascript	7.7 %	+0.5 %
6		C	7.1 %	-0.3 %
7	↓↓	C++	7.0 %	-0.6 %
8		Objective-C	4.4 %	-0.6 %
9	↑	R	3.4 %	+0.5 %
10	↓	Swift	3.0 %	+0.1 %
11		Matlab	2.6 %	-0.3 %
12		Ruby	2.1 %	-0.1 %
13		Visual Basic	1.5 %	-0.2 %
14		VBA	1.5 %	+0.0 %
15	↑	Scala	1.2 %	+0.4 %
16	↓	Perl	0.9 %	-0.1 %
17		lua	0.5 %	+0.0 %

Jan 2022

Worldwide, Jan 2022 compared to a year ago:				
Rank	Change	Language	Share	Trend
1		Python	28.74 %	-1.8 %
2		Java	18.01 %	+1.2 %
3		JavaScript	9.07 %	+0.6 %
4	↑	C/C++	7.4 %	+1.1 %
5	↓	C#	7.27 %	+0.7 %
6		PHP	6.06 %	+0.0 %
7		R	4.19 %	+0.3 %
8		Objective-C	2.27 %	-1.4 %
9		Swift	1.91 %	-0.2 %
10		TypeScript	1.74 %	-0.0 %
11		Matlab	1.74 %	+0.0 %
12		Kotlin	1.71 %	+0.0 %
13		Go	1.19 %	-0.1 %
14		VBA	1.14 %	-0.0 %
15		Ruby	1.07 %	-0.0 %
16		Rust	0.98 %	-0.0 %
17	↑↑↑	Ada	0.75 %	+0.1 %

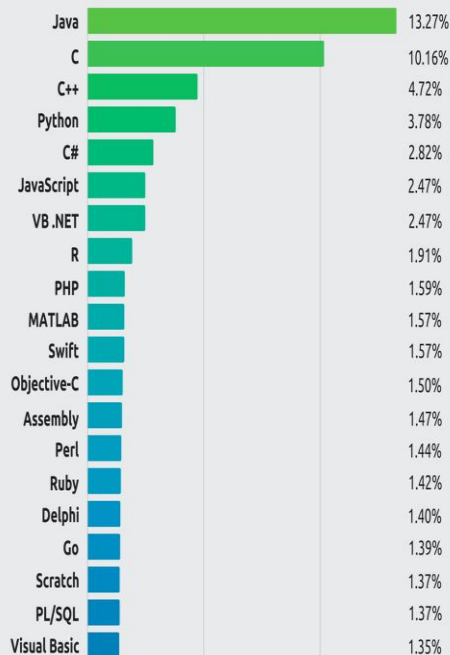
Jan 2023

Worldwide, Jan 2023 compared to a year ago:				
Rank	Change	Language	Share	Trend
1		Python	27.93 %	-0.9 %
2		Java	16.78 %	-1.3 %
3		JavaScript	9.63 %	+0.5 %
4	↑	C#	6.99 %	-0.3 %
5	↓	C/C++	6.9 %	-0.5 %
6		PHP	5.29 %	-0.8 %
7		R	4.03 %	-0.2 %
8	↑↑↑	TypeScript	2.79 %	+1.0 %
9		Swift	2.23 %	+0.3 %
10	↓↓	Objective-C	2.2 %	-0.1 %
11	↑↑	Go	1.94 %	+0.7 %
12	↑↑↑	Rust	1.9 %	+0.9 %
13	↓	Kotlin	1.81 %	+0.1 %
14	↓↓↓↓	Matlab	1.63 %	-0.1 %
15	↑	Ruby	1.13 %	+0.3 %
16	↓↓	VBA	1.03 %	-0.0 %

VERSO NUOVI LINGUAGGI – 2024

Top Programming Languages

Tiobe Index - December 2017



Jan 2022	Jan 2021	Change	Programming Language	Ratings
1	3	▲	 Python	13.58%
2	1	▼	 C	12.44%
3	2	▼	 Java	10.66%
4	4		 C++	8.29%
5	5		 C#	5.68%
6	6		 Visual Basic	4.74%
7	7		 JavaScript	2.09%
8	11	▲	 Assembly language	1.85%
9	12	▲	 SQL	1.80%
10	13	▲	 Swift	1.41%
11	8	▼	 PHP	1.40%
12	9	▼	 R	1.25%
13	14	▲	 Go	1.04%
14	19	▲	 Delphi/Object Pascal	0.99%
15	20	▲	 Classic Visual Basic	0.98%
16	16		 MATLAB	0.96%
17	10	▼	 Groovy	0.94%
18	15	▼	 Ruby	0.88%

Jan 2024	Jan 2023	Change	Programming Language
1	1		 Python
2	2		 C
3	3		 C++
4	4		 Java
5	5		 C#
6	7	▲	 JavaScript
7	10	▲	 PHP
8	6	▼	 Visual Basic
9	8	▼	 SQL
10	20	▲	 Scratch
11	12	▲	 Go
12	27	▲	 Fortran
13	17	▲	 Delphi/Object Pascal
14	15	▲	 MATLAB
15	9	▼	 Assembly language
16	11	▼	 Swift
17	25	▲	 Kotlin

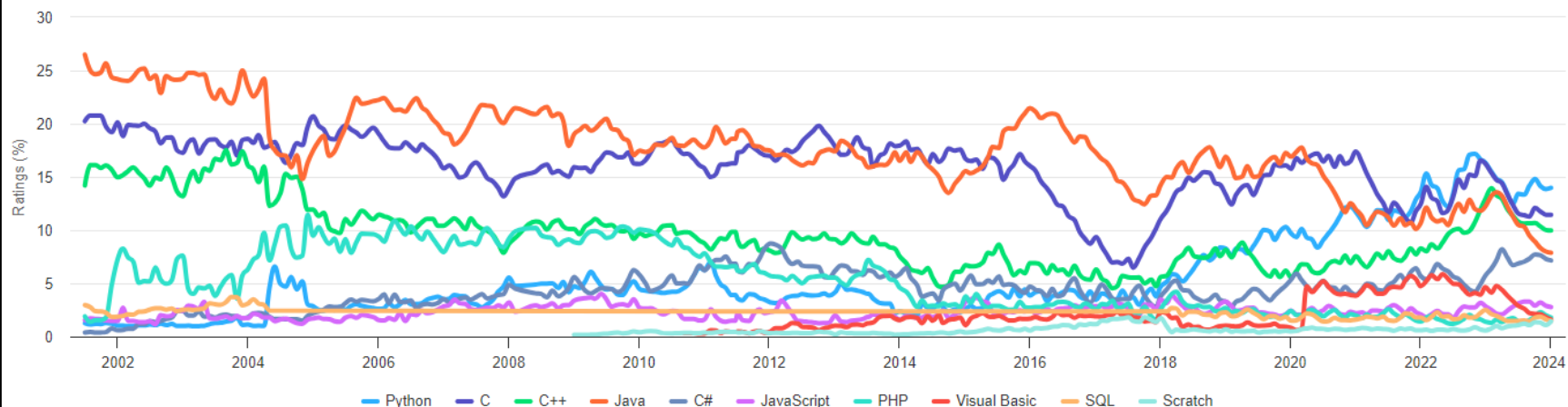
January Headline: C# is programming language of the year 2023!

Kotlin is the easy to learn/write competitor of Java.

VERSO NUOVI LINGUAGGI – 2024

TIOBE Programming Community Index

Source: www.tiobe.com



Programming Language	2024	2019	2014	2009	2004	1999	1994	1989
Python	1	4	8	6	11	22	22	-
C	2	2	1	2	2	1	1	1
C++	3	3	4	3	3	2	2	3
Java	4	1	2	1	1	16	-	-
C#	5	6	5	8	9	32	-	-
JavaScript	6	8	9	9	8	21	-	-



VERSO NUOVI LINGUAGGI – 2024

Language Rank	Types	Spectrum Ranking
1. Python		100.0
2. C		99.7
3. Java		99.5
4. C++		97.1
5. C#		87.7
6. R		87.7
7. JavaScript		85.6
8. PHP		81.2
9. Go		75.1
10. Swift		73.7

(dicembre 2019)

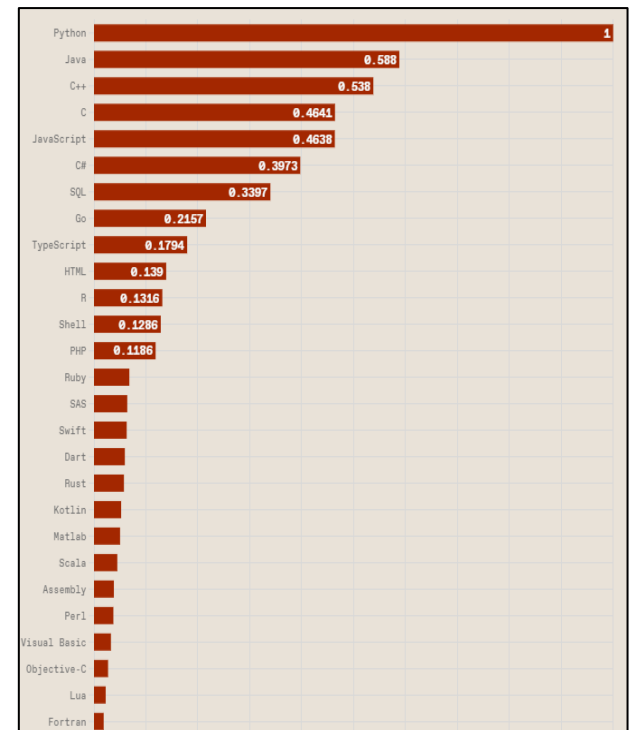
Language Rank	Types	Spectrum Ranking
1. Python		100.0
2. C		99.7
3. Java		99.5
4. C++		97.1
5. C#		87.7
6. R		87.7
7. JavaScript		85.6
8. PHP		81.2
9. Go		75.1
10. Swift		73.7

(dicembre 2023)

Classifica IEEE Spectrum
(dicembre 2021)

Language Ranking: IEEE Spectrum			
Rank	Language	Type	Score
1	Python		100.0
2	Java		95.4
3	C		94.7
4	C++		92.4
5	JavaScript		88.1
6	C#		82.4
7	R		81.7
8	Go		77.7
9	HTML		75.4
10	Swift		70.4
11	Arduino		68.4
12	Matlab		68.3

Classifica IEEE Spectrum
(dicembre 2023)





VERSO NUOVI LINGUAGGI – 2024

MA le classifiche vanno interpretate...

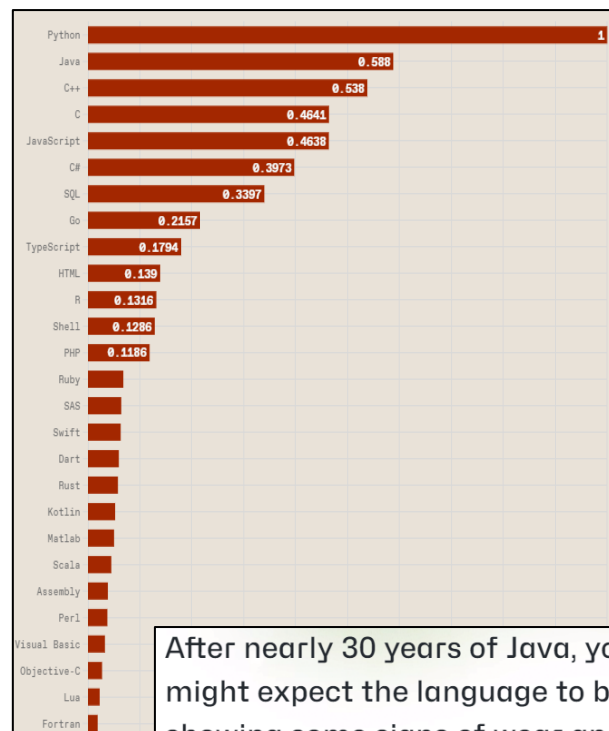
But Python alone does not make a career. In our “Jobs” ranking, it is SQL that shines at No. 1. Ironically though, you’re very unlikely to get a job as a pure SQL programmer. Instead, employers love, love, love, seeing SQL skills in tandem with some other language such as Java or C++. With today’s distributed architectures, a lot of business-critical data live in SQL databases, whether it’s the list of magic spells a player knows in an online game or the amount of money in their real-life bank account. If you want to do anything with that information, you need to know how to get at it.

But don’t let Python and SQL’s rankings fool you: Programming is still far from becoming a monoculture. Java and the various C-like languages outweigh Python in their combined popularity, especially for high-performance or resource-sensitive tasks where th

10 Most Demanding Programming Language In the Future

- #1 Java (Web/App and Software Development)
- #2 Python (Web Dev., 3D Rendering)
- #3 PHP (Developing Dynamic Websites)
- #4 JavaScript
- #5 C (Software & Application Language)
- #6 C++ (GUI Development & Graphics-based Software Language)
- #7 R (Complex Statistical Programming)
- #8 C# (.Net framework Language)
- #9 Swift (iOS and OS X Application Development Language)
- #10 Go (Web Applications, APIs, & Web Servers)

Classifica IEEE Spectrum
(dicembre 2023)



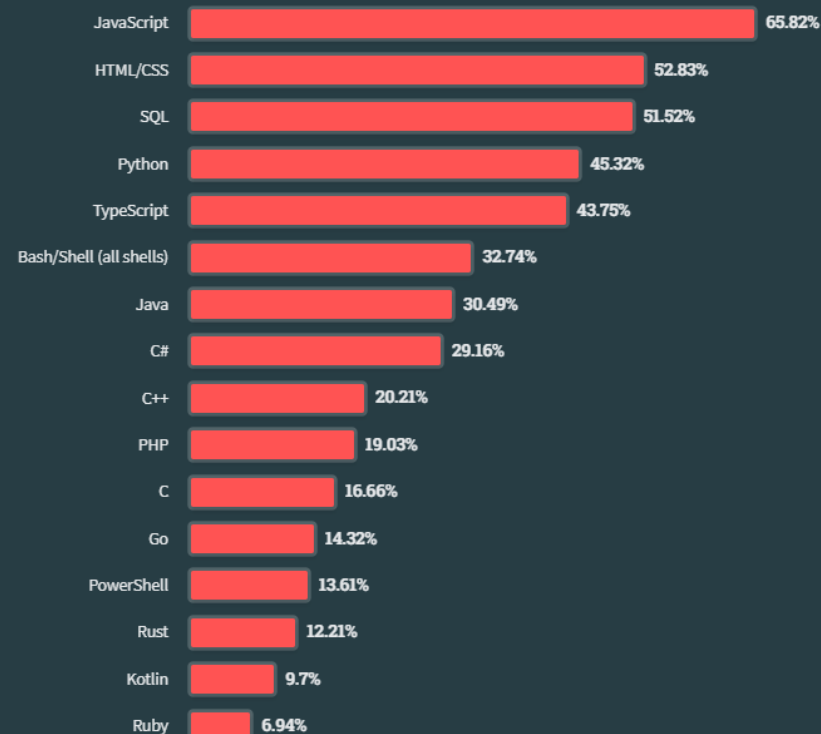
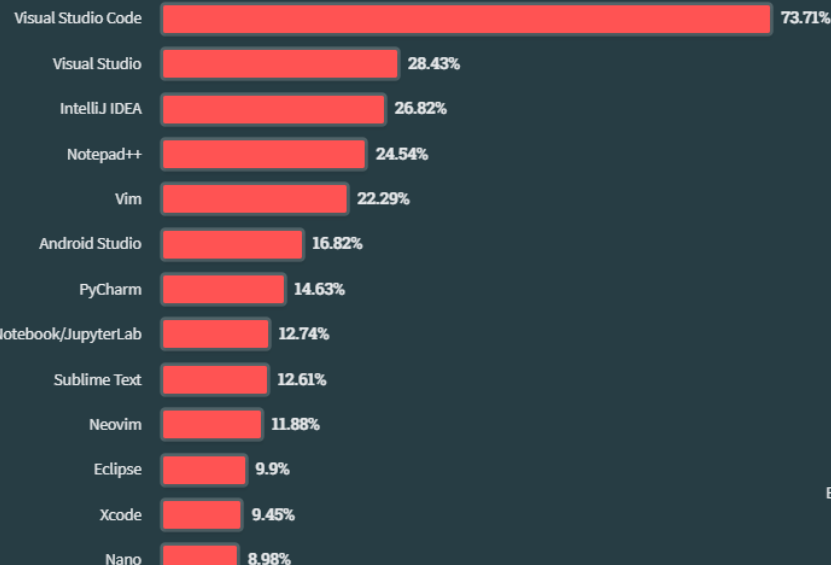
After nearly 30 years of Java, you might expect the language to be showing some signs of wear and tear, but nothing could be further from the truth. ”



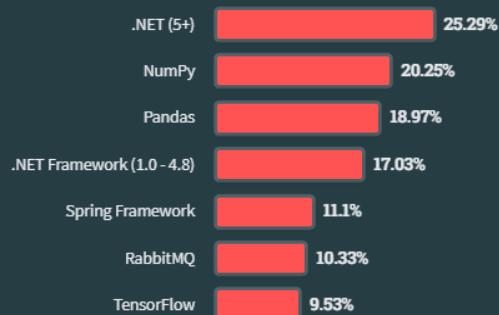
VERSO NUOVI LINGUAGGI – 2024

StackOverflow
Survey

2023
Developer
Survey



Other frameworks and libraries





NUOVI LINGUAGGI & NUOVE FIGURE

The 10 most in-demand tech jobs for 2023

Job	25th percentile	50th percentile	75th percentile
Cloud engineer	\$102,500	\$122,750	\$143,750
Database developer	\$105,000	\$127,250	\$144,250
DevOps engineer	\$108,250	\$133,750	\$157,500
Front-end developer	\$81,250	\$98,500	\$117,250
Help desk and desktop support professional	\$38,250 (tier 1); \$44,750 (tier 2); \$54,250 (tier 3)	\$46,250 (tier 1); \$54,500 (tier 2); \$65,750 (tier 3)	\$51,500 (tier 1); \$61,000 (tier 2); \$77,750 (tier 3)
Network administrator	\$82,500	\$101,000	\$118,500
Network security engineer	\$112,500	\$135,750	\$158,750
Software developer	\$103,250	\$126,500	\$148,000
Software engineer	\$106,500	\$127,000	\$149,750
Systems security manager	\$134,250	\$164,250	\$194,250

Fonte: Cio.com

Best Paying Jobs In Technology 2023

1. Cloud Architect
2. Computer Programmers
3. DevOps Engineer
4. Software Engineering Manager
5. Product Manager
6. Internet Of Things (IoT) Solutions Engineer
7. Blockchain Engineer
8. Big Data Engineer
9. Computer Support Specialists
10. Web Developers
11. Robotics Engineer
12. Database Administrators
13. Computer and Information Research Scientists
14. Mobile Applications Developer
15. Software Architect
16. Full-Stack Developer
17. Artificial Intelligence (AI) Engineer
18. Information Systems Security Manager
19. Data Security Analyst
20. Computer Systems Analysts
21. What is the Future of Jobs in Technology Sector?

Best Tech Jobs | Contents

- **Technology Careers List: A Wide Range of Opportunities**
 - Information Systems (IS) Manager
 - Software Developer
 - Information Security Analysts
 - Web Developer
 - Computer Programmer
 - Systems Analyst
 - Cybersecurity Analyst
- **Tech Jobs in Demand**
 - Cybersecurity Skills / Cybersecurity Analysts
 - AI & Machine Learning
 - Mobile Application Developers
- **Salaries for In-Demand Tech Jobs**
 - Artificial Intelligence Specialist
 - Software Developer
 - IT Manager
 - Mobile Designer
 - Web Developer/Designer
- **Tech Jobs of the Future**
- **Tech Career Path**

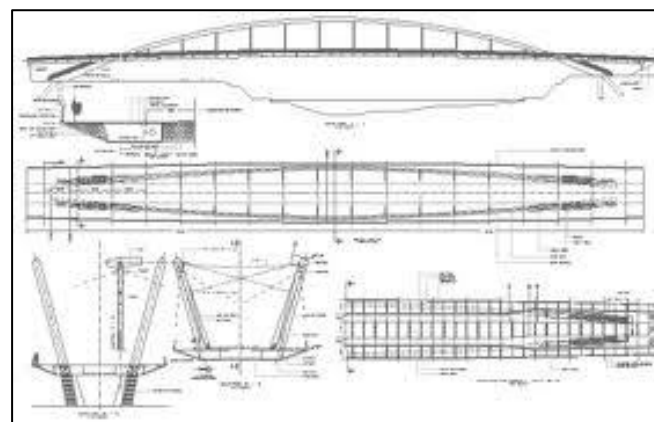
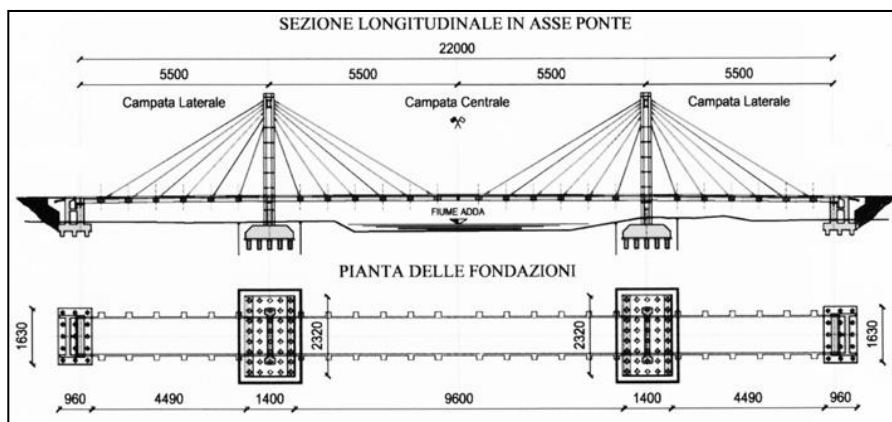
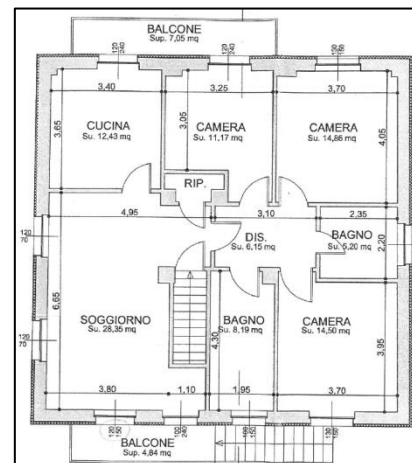
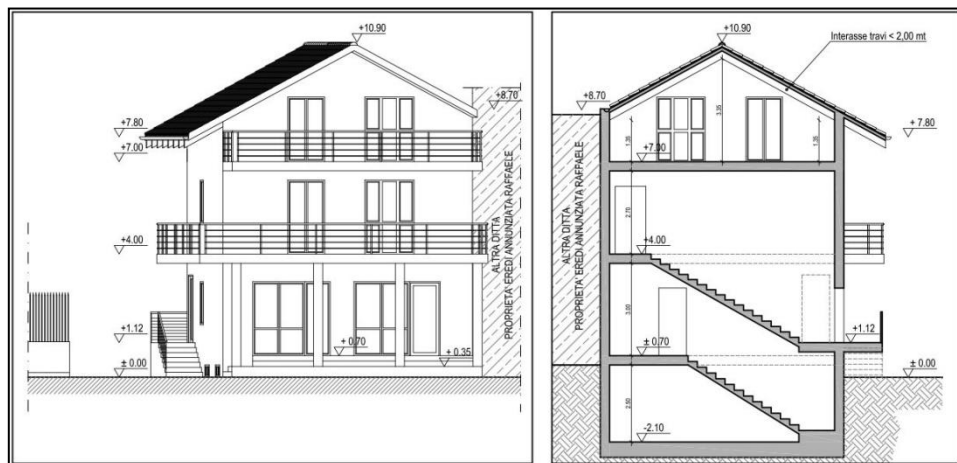


IL PROGETTO NON È IL CODICE

L'attività di progetto dev'essere riconosciuta come tale

- Il suo risultato – il **progetto** – dev'essere *un artefatto esistente e documentato in quanto tale*, come avviene da sempre in tutte le branche dell'ingegneria
 - il progetto di un sistema software non è il codice
 - esattamente come il *progetto di una casa* non è *la casa*, il *progetto di un ponte* non è *il ponte stesso*!
- Solitamente, nell'ingegneria il progetto è espresso tramite *disegni, tavole, diagrammi* che mirano a trasmettere:
 - la struttura d'insieme
 - le parti coinvolte e le loro relazioni

IL PROGETTO NON È IL CODICE





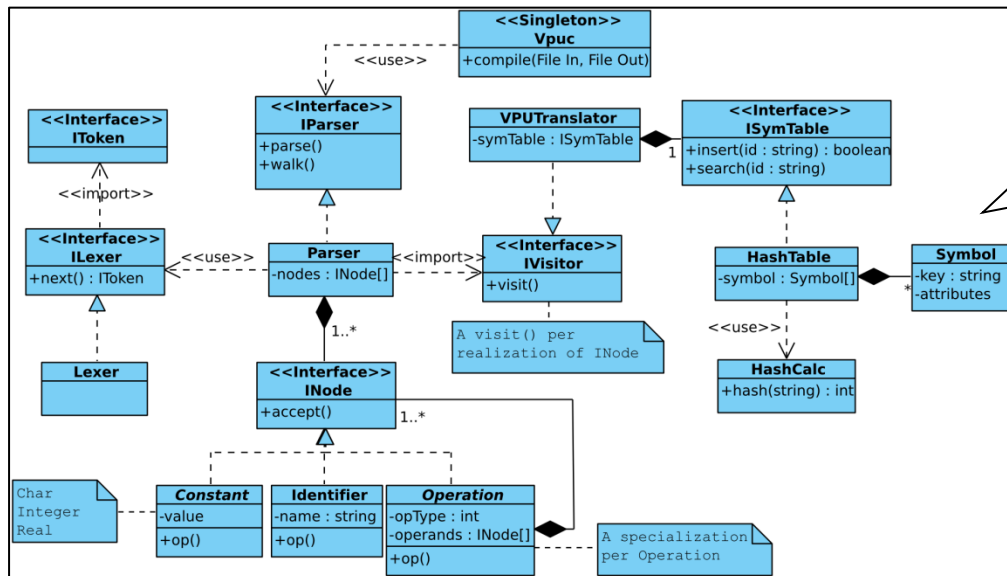
IL PROGETTO NON È IL CODICE

L'attività di progetto dev'essere riconosciuta come tale *anche nell'ingegneria informatica*

- L'attività di progetto dev'essere precedente e totalmente disaccoppiata dall'implementazione
- La relativa "giovinezza" dell'ingegneria informatica ha portato molti "artigiani del fai da te" e pensare erroneamente di poter far senza questi strumenti..
- ..ma non si gestisce la complessità senza poter operare a *un adeguato livello di astrazione – che non è il codice*
- *Servono metodologie, standard e linguaggi grafici per esprimere il progetto di un sistema software*

ESEMPIO: PROGETTO IN UML

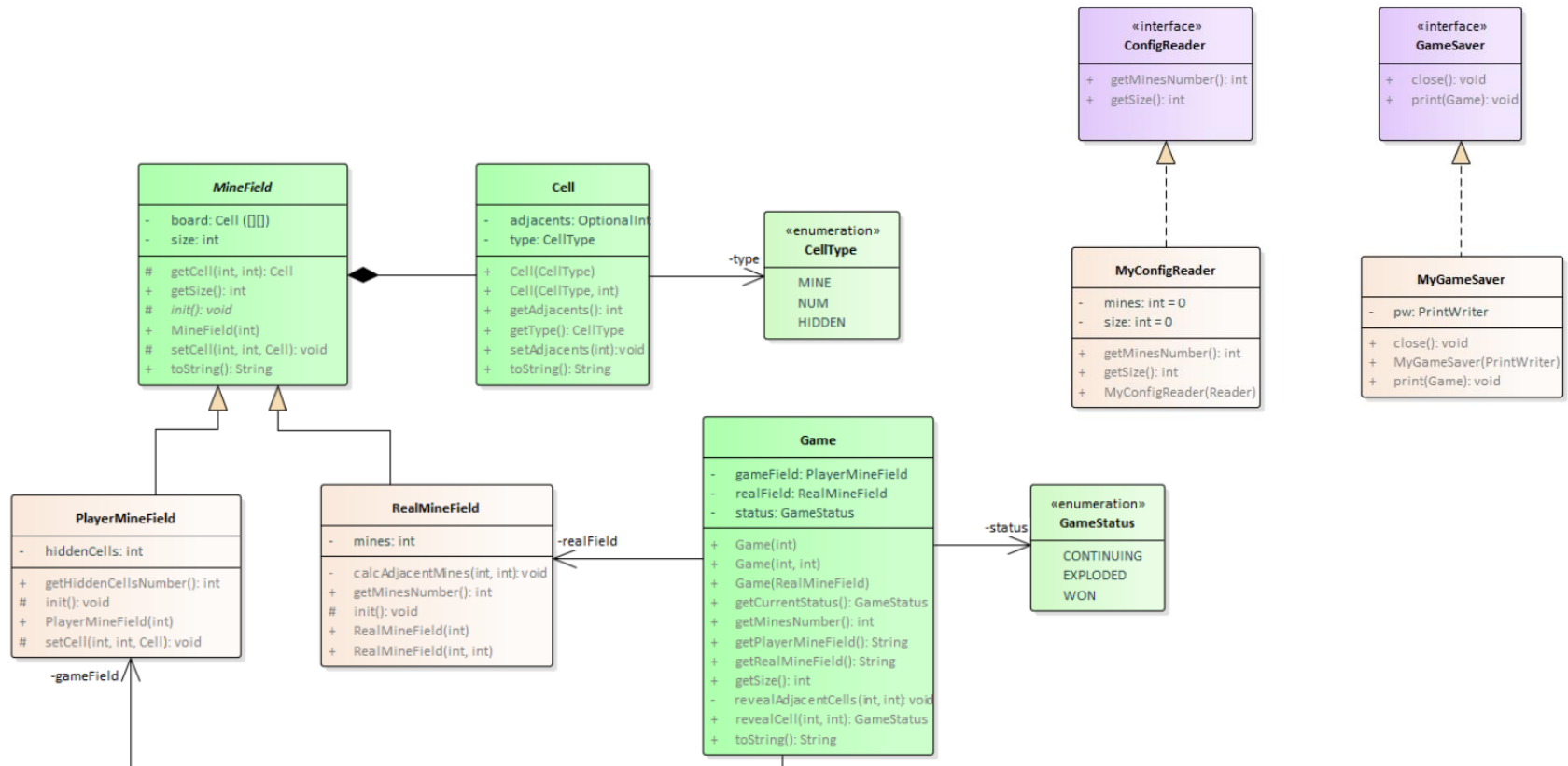
- **UML (Unified Modelling Language)** è uno standard internazionale per esprimere modelli di sistemi (non necessariamente informatici)
- Fra i molti diagrammi che UML introduce, il *diagramma di struttura* esprime la *struttura di un sistema a oggetti*



Esempio di progetto di un sistema di media complessità in UML

ESEMPIO: PROGETTO IN UML

- ESEMPIO: parte di un diagramma da un compito d'esame





SOLO NUOVI LINGUAGGI..?

MA un buon linguaggio non basta..

- Per avere successo occorre *non lasciarlo solo*
- Intorno a esso deve essere previsto un *ecosistema* pensato secondo una *precisa visione infrastrutturale*
 - l'esperienza (C++..) insegna che poter creare propri componenti e oggetti è importante, ma *standardizzare il linguaggio non basta*
- Non si può lasciare che ogni azienda (o strumento) realizzi "a proprio gusto" *tutto quello che ci sta intorno*
 - se ognuno realizza le proprie librerie (circa simili, ma non identiche) si creano *entità e comunità incomunicabili*
 - se ognuno copre solo alcune piattaforme, si finisce per *rifare le stesse cose N volte, con spreco di tempo e denaro*



DAI LINGUAGGI ALLE INFRASTRUTTURE

Occorre una *infrastruttura standard*

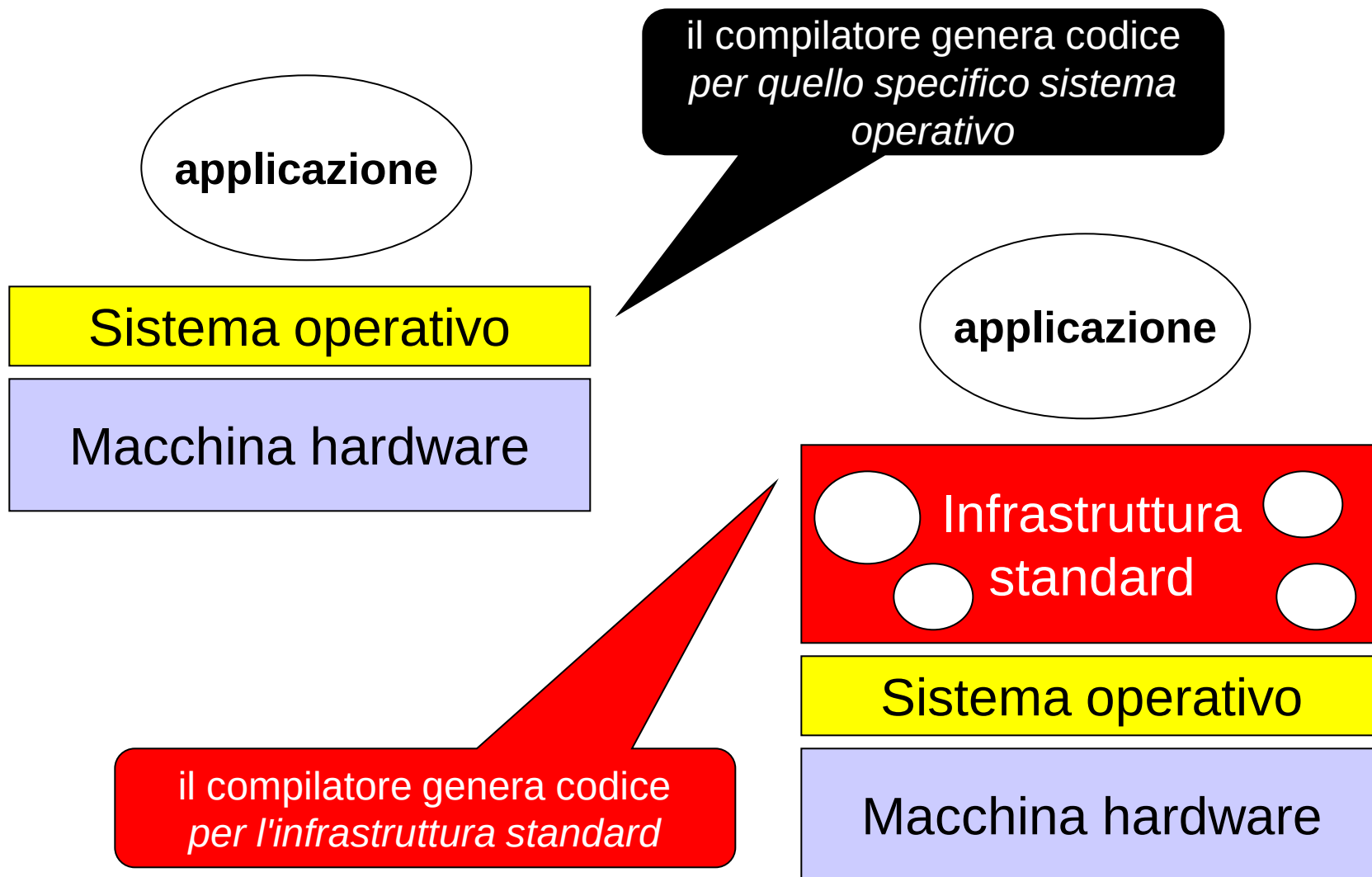
- un ambiente che offra *componenti pronti standardizzati*, certamente presenti su ogni installazione
- uno strato di sicura affidabilità su cui contare e costruire
- disponibile per più sistemi operativi e diversi hardware
- uno strato che *renda indipendente chi sta sopra dalla specifica configurazione hardware/software sottostante*

Se ci si riesce, *cambia totalmente lo scenario*

- cambia il processo di compilazione ed esecuzione
- *cambia il modo stesso di vedere "il software"*
- il sogno di ogni softwarista: *write once, run everywhere*



DAI LINGUAGGI ALLE INFRASTRUTTURE





DAI LINGUAGGI ALLE INFRASTRUTTURE

- Poiché costruire software richiede tempo e risorse, si cerca di *massimizzare la resa* di tale investimento
- Perciò il software dovrebbe essere il più possibile *portabile e inter-piattaforma*
- Per questo interessano **INFRASTRUTTURE PORTABILI e MULTI-PIATTAFORMA**
 - installabili su diversi sistemi hardware
 - con diversi sistemi operativi e configurazioni software
 - in grado di offrire alle applicazioni costruite sopra di esse un *ambiente noto, ricco e dal funzionamento uniforme*

INFRASTRUTTURE MULTI-PIATTAFORMA

applicazione

Se il compilatore genera codice *per l'infrastruttura* (e non per lo specifico sistema operativo...)



Sistema operativo

Macchina hardware

applicazione



SO 1

HW

SO 2

HW

SO x

HW

..si diventa *indipendenti* sia dal sistema operativo, sia dall'hardware sottostante!

COMPILAZIONE ED ESECUZIONE

applicazione

PRIMA il compilatore generava codice oggetto, lo *collegava (link)* **STATICAMENTE** alle librerie e produceva un eseguibile **AUTOCONTENUTO**

Sistema operativo

Macchina hardware

applicazione

ORA il compilatore genera codice *nel formato intermedio dell'infrastruttura* e non collega librerie staticamente: *si caricano e collegano DINAMICAMENTE, a run-time, i componenti che servono*

Infrastruttura
standard

SO 1

SO 2

SO x

HW

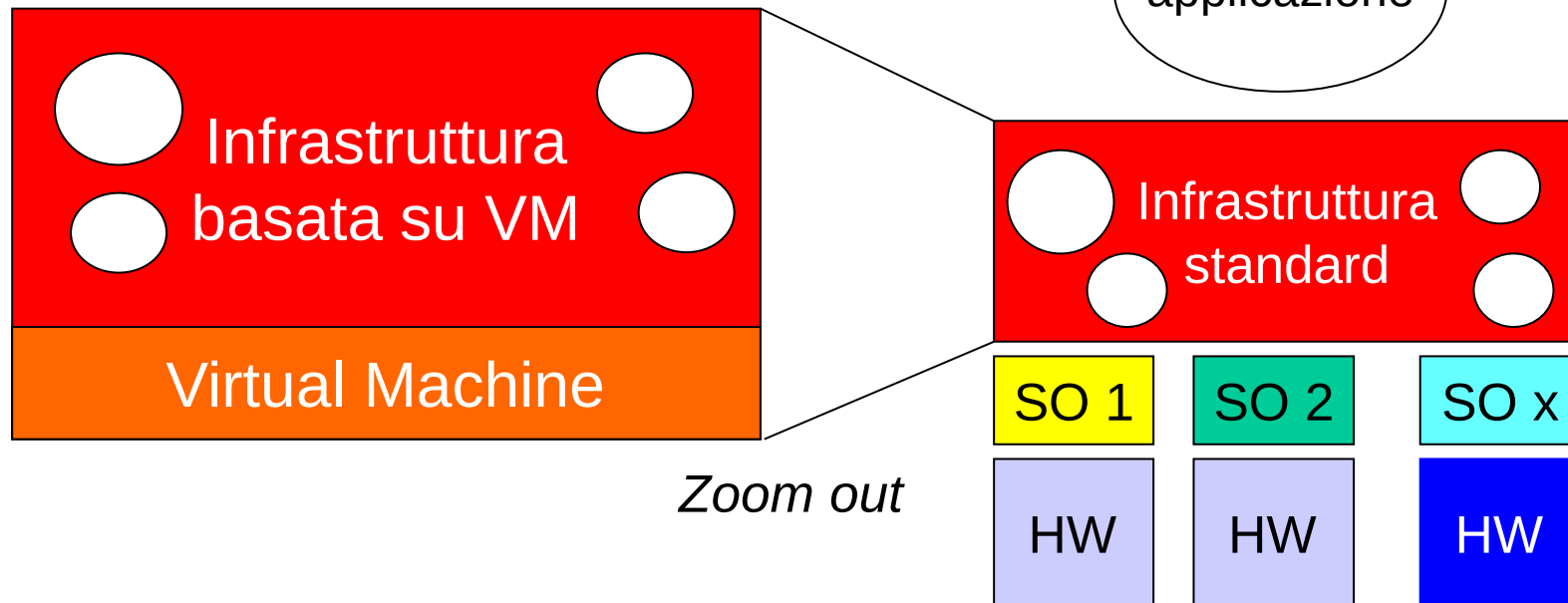
HW

HW



INFRASTRUTTURE MULTIPLE MULTI-PIATTAFORMA

Se poi l'infrastruttura a sua volta si appoggia su uno strato-base di **MACCHINA VIRTUALE**, che astrae da ciò che c'è sotto...



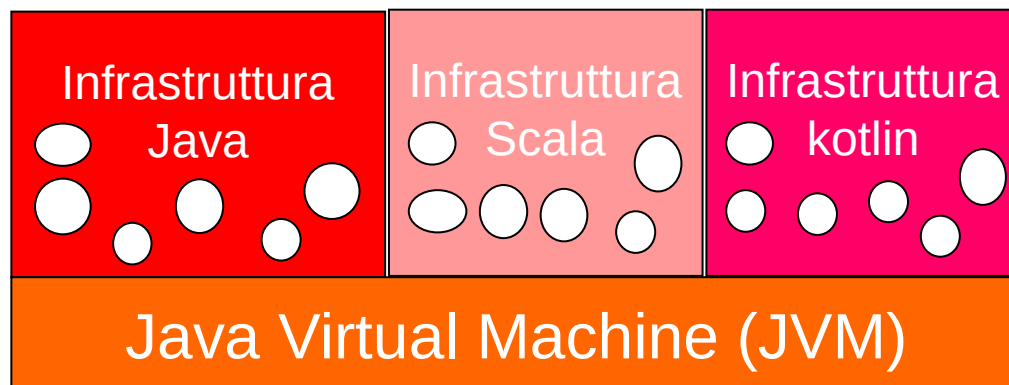


INFRASTRUTTURE MULTIPLE MULTI-PIATTAFORMA

Se poi l'infrastruttura a sua volta si appoggia su uno strato-base di **MACCHINA VIRTUALE**, che astrae da ciò che c'è sotto...



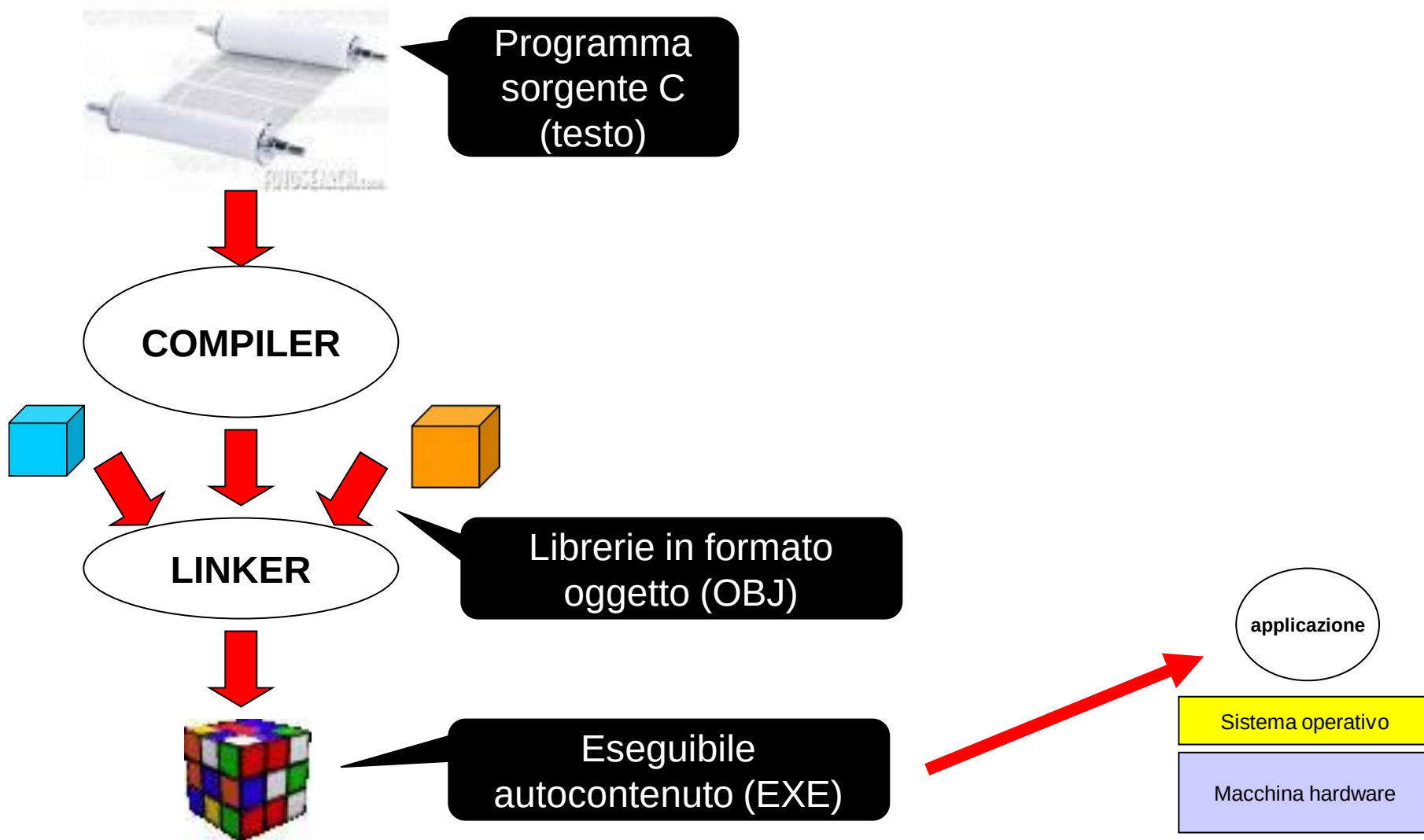
...si possono avere anche *DIVERSE INFRASTRUTTURE* sulla stessa VM, in grado di *interagire fra loro!*



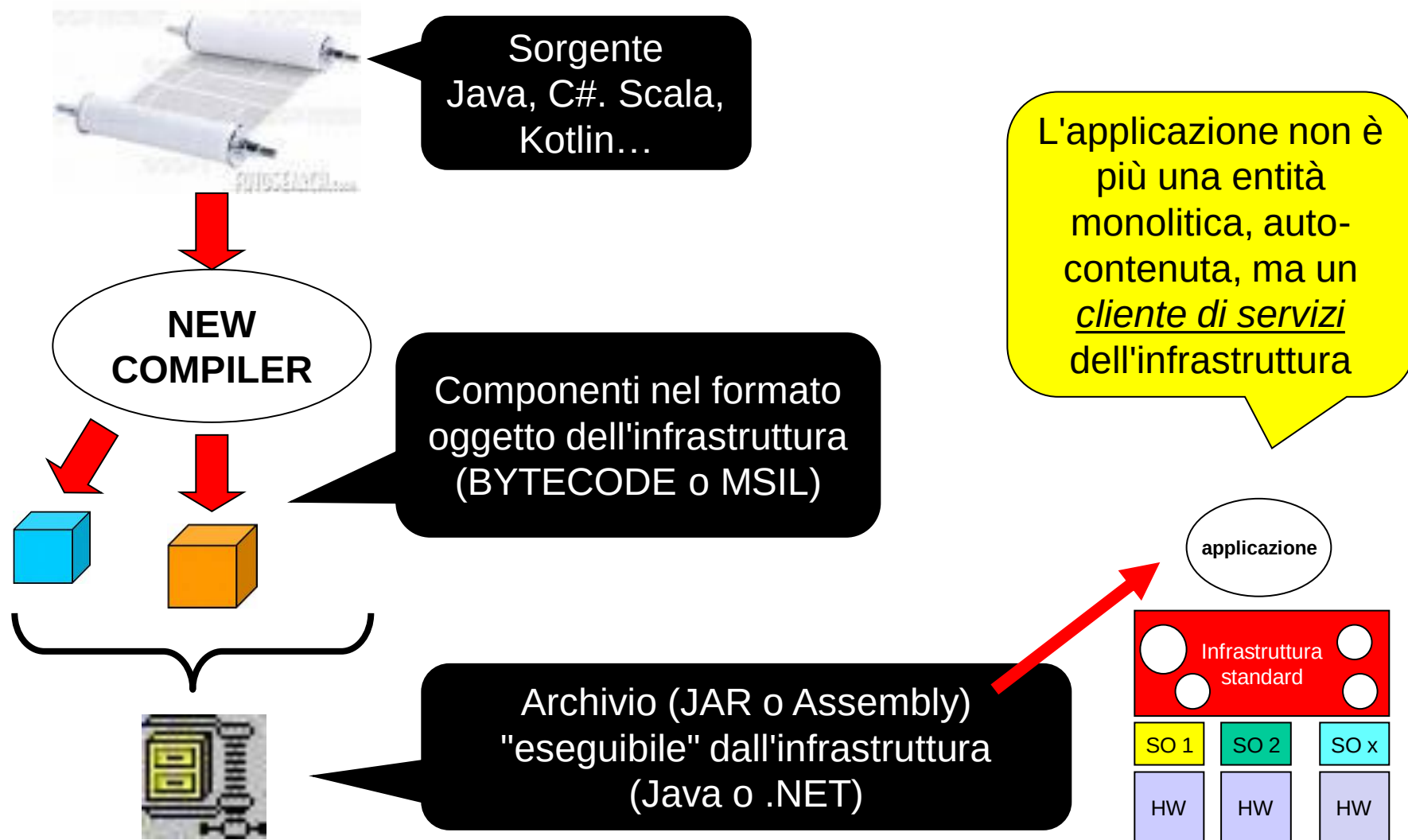
RIASSUMENDO

- Cambia il *processo di costruzione* del software
 - l'obiettivo non è più generare un programma per un certo processore e versione di sistema operativo..
 - ..ma generare qualcosa di *massimamente riusabile*
- Cambia il *processo di esecuzione* del software
 - non si esegue più un eseguibile (EXE) prodotto apposta per un certo processore e sistema operativo..
 - ..ma si fa eseguire *all'infrastruttura* un "prodotto" che non sarebbe eseguibile dal sistema operativo stand alone, poiché non è "autocontenuto": *un eseguibile portabile, inter-piattaforma*
- Si paga un (piccolo) prezzo per questo passo extra, ma i *vantaggi sono molto superiori al costo*
 - processo di sviluppo adatto a un *mondo distribuito e in divenire*

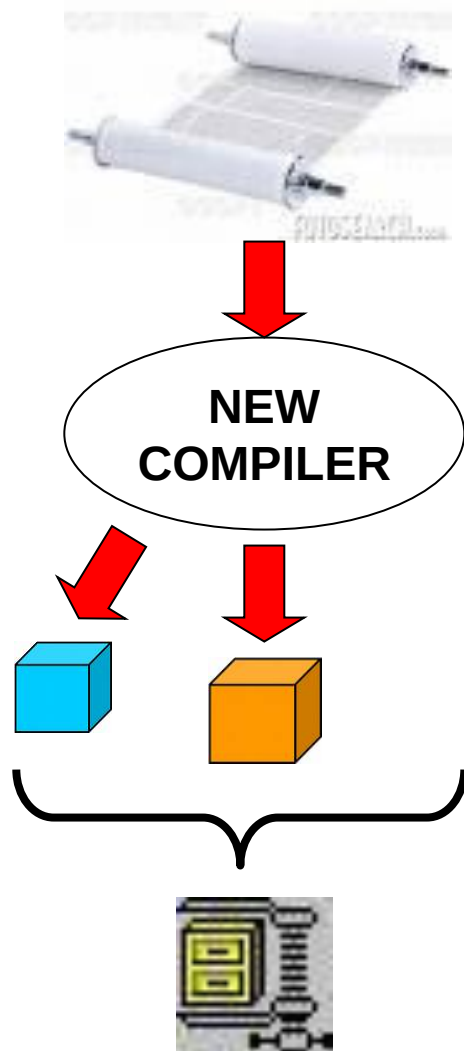
DAL "MONDO C" ...



... AL "NUOVO MONDO"



IL MONDO DI JAVA e C#

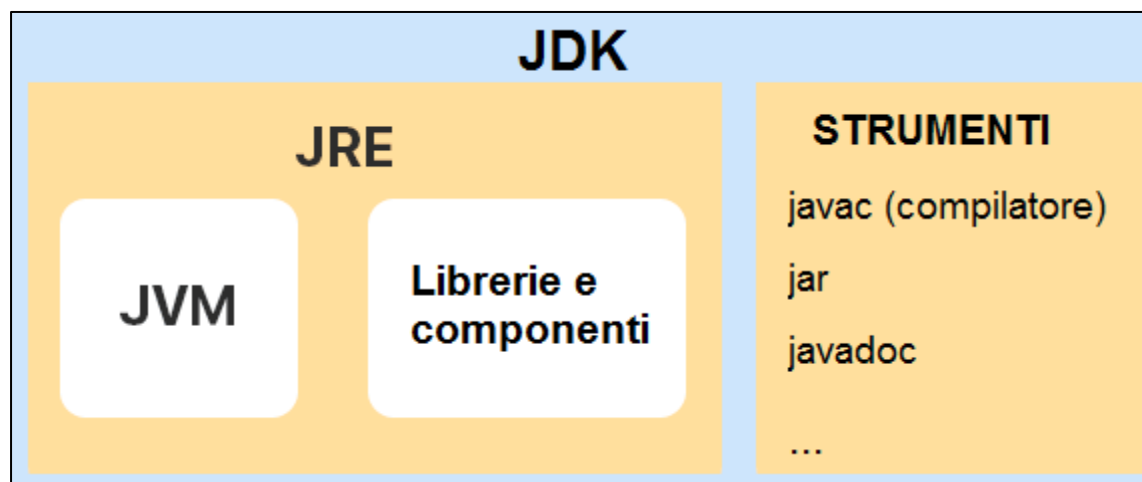


- Il sorgente è strutturato in una serie di componenti, i più importanti dei quali si chiamano **classi**
 - in Scala e kotlin, anche *oggetti (singleton)*
- Ogni componente viene compilato in un **formato portabile**, indipendente da hw e sistema operativo
- L'eseguibile può essere compilato su una sistema operativo **e funzionare su un altro**, purché la macchina sia **dotata della stessa infrastruttura**
- Per consentire ciò, cambia il senso di "eseguibile"
 - non è più un "eseguibile" in senso classico
 - un EXE di ".NET" o un JAR di Java non funziona su una macchina "nuda", col solo sistema operativo
- Occorre uno **STRATO-PONTE nell'infrastruttura**, che **interpreti** il formato portabile adattandolo alla specifica macchina
 - È l'unico strato *dipendente dalla piattaforma*.



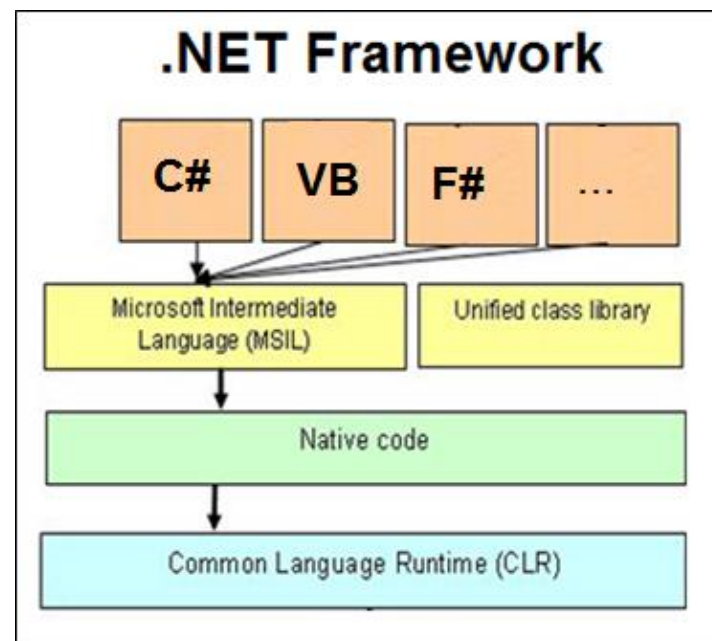
IL RUNTIME NELL'INFRASTRUTTURA

- Lo strato infrastrutturale di **Java** si chiama **JRE** (*Java Runtime Environment*)
 - due livelli: Java Virtual Machine (JVM) + JRE vero e proprio
 - per *eseguire* applicazioni Java dev'essere installato sul computer
 - leggero, gratuito, multi-piattaforma (Windows, Mac, Linux, Android*)
 - per *sviluppare* applicazioni serve un set di strumenti più ampio, che include compilatore e altri strumenti: il **JDK** (*Java Development Kit*)



IL RUNTIME NELL'INFRASTRUTTURA

- Lo strato infrastrutturale di **C#** si chiama **.NET**
 - articolazione simile: **Common Language Runtime (CLR)** + altro
 - **pre-installato** come **.NET framework** nei sistemi Windows (7, 10 ...) *incluso (fino alla versione 4.0) il compilatore a riga di comando (csc)*
 - versione multiplatforma anche per MacOS, Linux, Docker (<https://dotnet.microsoft.com>)
 - progetto open source, **MONO** (<https://www.mono-project.com/>)
 - molteplicità di linguaggi supportati
- Versione attuale .NET 8.0



LA PIATTAFORMA JAVA

Scaricamento e installazione (dettagli in slide *Strumenti*)



Workshop
[OpenJDK FAQ](#)
[Installing](#)
[Contributing](#)
[Sponsoring](#)
[Developers' Guide](#)
[Vulnerabilities](#)
[Mailing lists](#)
[IRC - Wiki](#)
[Bylaws - Census](#)
[Legal](#)

JEP Process

Source code
[Mercurial](#)
[GitHub](#)

Groups
[\(overview\)](#)
[2D Graphics](#)
[Adoption](#)
[AWT](#)
[Build](#)
[Compatibility & Specification](#)
[Review](#)
[Compiler](#)
[Conformance](#)
[Core Libraries](#)
[Governing Board](#)
[HotSpot](#)
[IDE Tooling & Support](#)
[Internationalization](#)
[JMX](#)
[Members](#)
[Networking](#)
[Porters](#)
[Quality](#)
[Security](#)
[Serviceability](#)
[Sound](#)
[Swing](#)
[Vulnerability](#)
[Web](#)

Projects
[\(overview\)](#)
[Amber](#)
[Annotations Pipeline](#)

What is this? The platform implementation Edition, and related projects

Download and install popular Linux distributions licensed, production-ready are at jdk.java.net/11 licensed JDK 15 binaries Windows, based on the

Learn how to use the wide range of environments

Hack on the JDK its Community: Browse the Mercurial repository to contribute a patch to a component, or define

OpenJDK JDK 21.0.2 General-Availability Release

This page provides production-ready open-source builds of the Java Development Kit, version 21, an implementation of the Java SE 21 Platform under the GNU General Public License, version 2, with the Classpath Exception.

Commercial builds of JDK 21.0.2 from Oracle, under a non-open-source license, can be found [here](#).

Documentation

- Features
- Release notes
- API Javadoc

Builds

Linux/AArch64	tar.gz (sha256)	201827024 bytes
Linux/x64	tar.gz (sha256)	203616881
macOS/AArch64	tar.gz (sha256)	197537405
macOS/x64	tar.gz (sha256)	199891097
Windows/x64	zip (sha256)	201328186

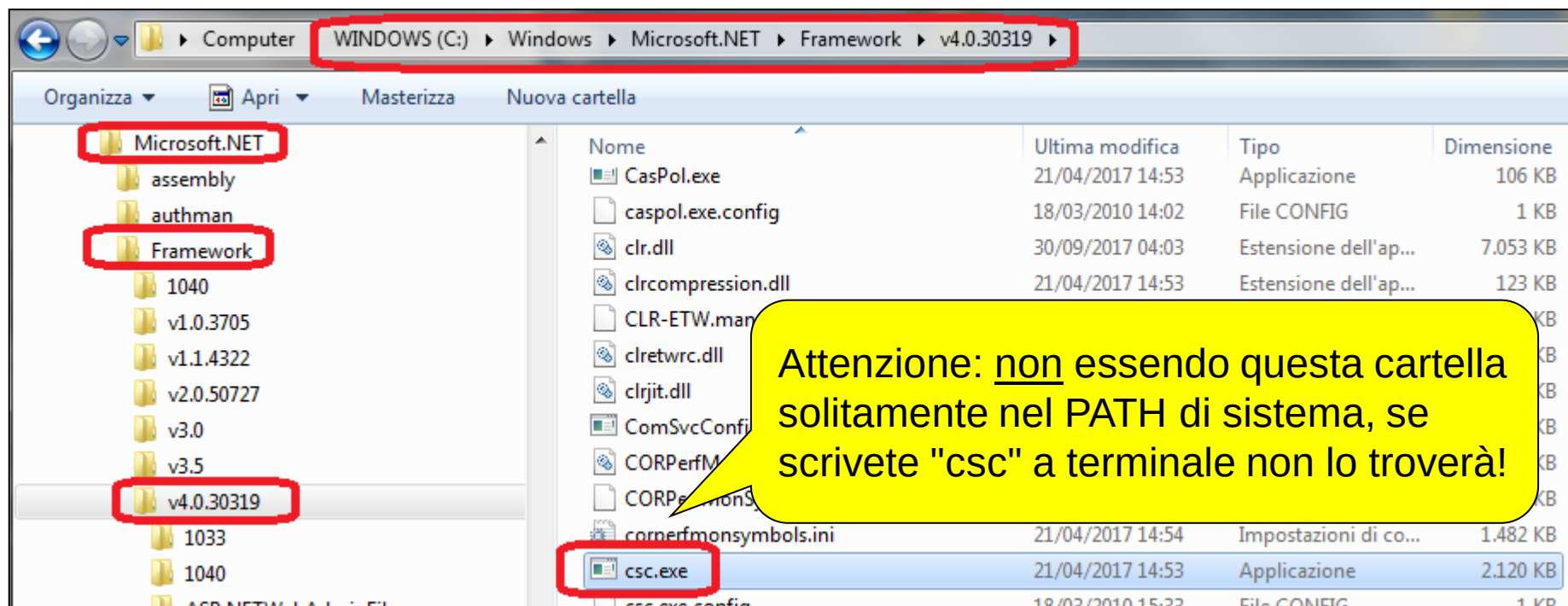
Notes

- If you have difficulty downloading any of these files please contact download-help@openjdk.org.

Scegliere la giusta piattaforma

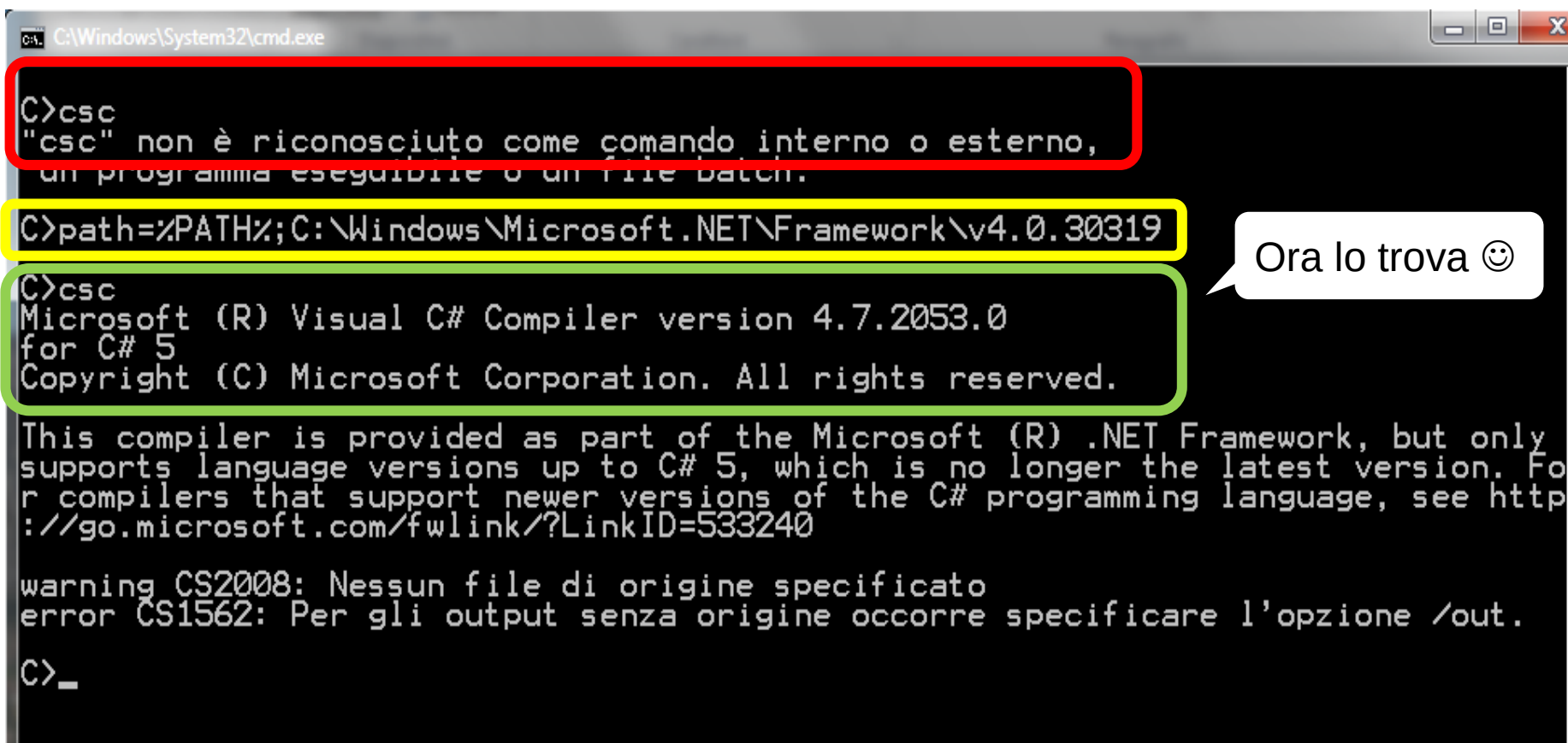
LA PIATTAFORMA .NET

- Il compilatore (vecchio) da riga di comando è nella cartella del .NET Framework (una sottocartella di C:\Windows)
 - potreste averne anche più versioni: 3, 3.5, 4.0, ...
 - es. C:\Windows\Microsoft.NET\Framework\v4.0.30319 [e/o Framework64]



LA PIATTAFORMA .NET

- Per usarlo basta aggiungere la cartella al path corrente:



```
C:\Windows\System32\cmd.exe

C>csc
"csc" non è riconosciuto come comando interno o esterno,
un programma eseguibile o un file batch.

C>path=%PATH%;C:\Windows\Microsoft.NET\Framework\v4.0.30319

C>csc
Microsoft (R) Visual C# Compiler version 4.7.2053.0
for C# 5
Copyright (C) Microsoft Corporation. All rights reserved.

This compiler is provided as part of the Microsoft (R) .NET Framework, but only
supports language versions up to C# 5, which is no longer the latest version. Fo
r compilers that support newer versions of the C# programming language, see http
://go.microsoft.com/fwlink/?LinkID=533240

warning CS2008: Nessun file di origine specificato
error CS1562: Per gli output senza origine occorre specificare l'opzione /out.

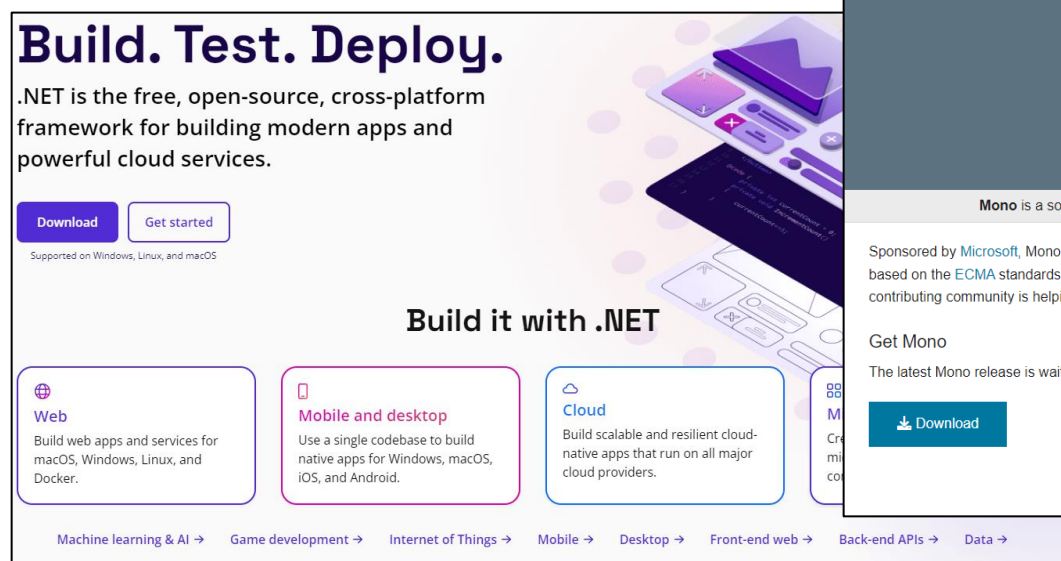
C>_
```

Ora lo trova 😊



LA PIATTAFORMA .NET

- Dal 2019 il **.NET Framework** (solo per Windows) è stato sostituito dalla **piattaforma .NET** (ora versione **8.0**)
 - allineato a C# 12.0 e Visual Studio 2023
 - sito: <https://dotnet.microsoft.com>
- Vedi anche progetto open source **Mono**



Build. Test. Deploy.

.NET is the free, open-source, cross-platform framework for building modern apps and powerful cloud services.

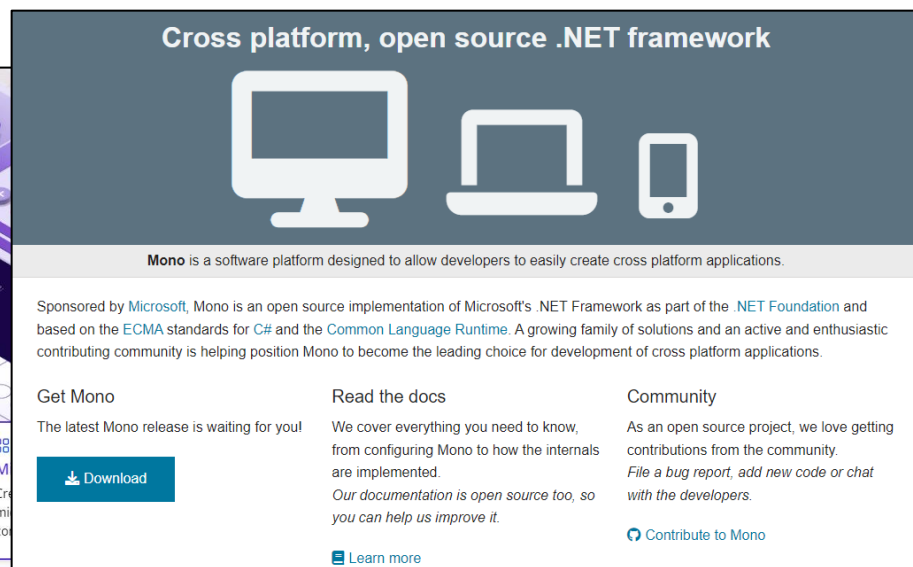
[Download](#) [Get started](#)

Supported on Windows, Linux, and macOS

Build it with .NET

- Web**
Build web apps and services for macOS, Windows, Linux, and Docker.
- Mobile and desktop**
Use a single codebase to build native apps for Windows, macOS, iOS, and Android.
- Cloud**
Build scalable and resilient cloud-native apps that run on all major cloud providers.

Machine learning & AI → Game development → Internet of Things → Mobile → Desktop → Front-end web → Back-end APIs → Data →



Cross platform, open source .NET framework

Mono is a software platform designed to allow developers to easily create cross platform applications.

Sponsored by [Microsoft](#), Mono is an open source implementation of Microsoft's .NET Framework as part of the [.NET Foundation](#) and based on the [ECMA](#) standards for [C#](#) and the [Common Language Runtime](#). A growing family of solutions and an active and enthusiastic contributing community is helping position Mono to become the leading choice for development of cross platform applications.

Get Mono
The latest Mono release is waiting for you!
[Download](#)

Read the docs
We cover everything you need to know, from configuring Mono to how the internals are implemented.
Our documentation is open source too, so you can help us improve it.
[Learn more](#)

Community
As an open source project, we love getting contributions from the community.
File a bug report, add new code or chat with the developers.
[Contribute to Mono](#)



LA PIATTAFORMA .NET

- Dal 2019 il **.NET Framework** (solo per Windows) è stato sostituito dalla **piattaforma .NET** (ora versione **8.0**)
 - allineato a **C# 12.0** e **Visual Studio 2023**
 - Sito: <https://dotnet.microsoft.com>

Download .NET

For Windows

.NET 8.0

Long Term Support

.NET SDK x64

Version 8.0.1, released January 9, 2024

[All .NET 8.0 downloads](#) [All .NET versions](#)

Enhance your .NET experience

Explore further tools

Visual Studio

Fully-featured integrated development environment (IDE) on Windows for building every type of .NET application.

Visual Studio Code

Develop on Linux, macOS, or Windows to build cross-platform websites and services. Install the **C# extension** to get the best experience.

Visual Studio for Mac

Build native Android, iOS, macOS, and Windows apps with .NET MAUI, plus websites and services with ASP.NET Core.

Other .NET downloads

.NET Framework

Looking for previous .NET Framework downloads for Windows?

[Download .NET Framework →](#)

.NET Coding Pack

With one download, the .NET Coding Pack gives you everything you need to get started coding with C#. The pack includes the VS Code editor, the .NET SDK, Interactive Notebooks, and more!

[Download .NET Coding Pack →](#)

Docker

Find official images for .NET and ASP.NET Core on the Microsoft Artifact Registry.

[Explore .NET images →](#)



AMBIENTI ONLINE

Per sperimentare senza dover installare niente sul proprio computer (o usando un tablet o smartphone), si possono usare gli *ambienti online*

- disponibili per praticamente ogni linguaggio
- accessibili con un comune browser
- spesso chiamati «**playground**»
 - Java (Kotlin, Scala, Swift..): <https://code.labstack.com/java>
 - Java: <https://www.studytonight.com/code/playground/java/>
 - C# playground: <https://dotnetfiddle.net/srx9kM>
 - Kotlin playground: <https://play.kotlinlang.org/>
 - Scala: <https://scastie.scala-lang.org/>
<https://scalafiddle.io/>
<https://www.katacoda.com/courses/scala/playground>
 - ...

AMBIENTI ONLINE

.NET Fiddle New Fork Run Share

Options Options

Language: C#

Project Type: Console

Compiler: .NET 4.7.2

C# PlayGround by Alen Vadassery

```
1 using System;
2
3 public class Esem
4 public static vo
5     float f = (fl
6     System.Consol
7 }
8 }
```

0.8660254

Kotlin Kotlin Playground is an online sandbox to explore Kotlin programming language. Browse code samples directly in the browser

```
public object Contatore {
    private var stato: Int = 0;
    public fun reset(): Unit {stato=0;}
    public fun inc():Unit {stato++;}
    public fun getValue(): Int {return stato;}
}

fun main(args: Array<String>) {
    Contatore.reset(); Contatore.inc();
    System.out.println(Contatore.getValue());
}
```

Scastie Editor Run New Format Clear Annotations Worksheet Save

Build Settings

object Contatore {

i You don't need a main method (or extends Scastie) in Worksheet Mode

```
private var stato: Int = 0;
def reset(): Unit = {stato=0;}
def inc():Unit = {stato+=1;}
def getValue(): Int = {return stato;}
}

object MyMain {
    def main(args: Array[String]):Unit = {
        Contatore.reset(); Contatore.inc();
        println(Contatore.getValue());
    }
}
```

1

trinket Java beta Run Share

New Tutorials: NUMPY TKINTER KOTLIN JAVASCRIPT

Java Compiler:

Main.java

```
1 /*
2 We have created a Main class for you
3 */
4 public class Main {
5     public static void main(String args[]) {
6         /*
7         Main method...
8         */
9     }
10 }
```

labstack Untitled Notes

Java (11.0)

```
1 public class Code {
2     public static void main(String[] args) {
3         System.out.println("Hello from Java!");
4     }
5 }
```

Input

Output

codingground Compile and Execute Scala Online (Scala v2.10.6)

Execute Share HelloWorld.scala STDIN Result

```
1 object Contatore {
2     private var stato: Int = 0;
3     def reset(): Unit = {stato=0;}
4     def inc(): Unit = {stato+=1;}
5     def Value(): Int = {return stato;}
6 }

7 def main {
8     args: Array[String] : Unit = {
9         re.reset(); Contatore.inc();
10        tln(Contatore.getValue());
11    }
```

\$scalac *.scala



LA TIPICA OBIEZIONE (1/2)

MA.. *così non ho un “vero eseguibile”!*

- Occhio alle parole: cosa vuol dire “*eseguibile*” ?
- Ogni file “eseguibile” lo è solo *nel contesto di una certa infrastruttura*, che gli fornisce il necessario contorno
 - un “eseguibile” Windows non funziona in MacOS
 - un “eseguibile” .NET non funziona in Windows senza .NET
 - un file Word non si apre senza Microsoft Word installato..
- Una entità è “eseguibile” (apribile) solo se è presente lo strato software che la interpreta, *non perché il suo nome finisce per “EXE”*
 - un JAR di Java è dunque “eseguibile” quanto un EXE di C#: ciò che conta è che sia presente l'infrastruttura di supporto



LA TIPICA OBIEZIONE (2/2)

Sarà anche vero, ma... *gli eseguibili “veri” si aprono facendoci sopra “doppio clic”!*

E INFATTI...

- con la giusta configurazione del JRE (vedere slide!) si rendono eseguibili con doppio clic le applicazioni Java (Scala, Kotlin) distribuite come **archivio JAR**
 - Android: archivio APK
- l'installazione del .NET framework rende eseguibili le applicazioni .NET distribuite come **archivio EXE**
 - Occhio al nome: gli "EXE di .NET *non sono* i classici eseguibili di Windows... tanto è vero che su Windows "puro" non girano!
 - con gli EXE "classici" hanno in comune solo la desinenza.

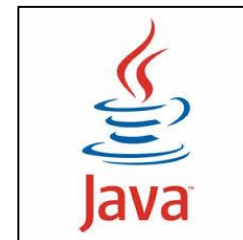


JAVA e C#: L'INFRASTRUTTURA

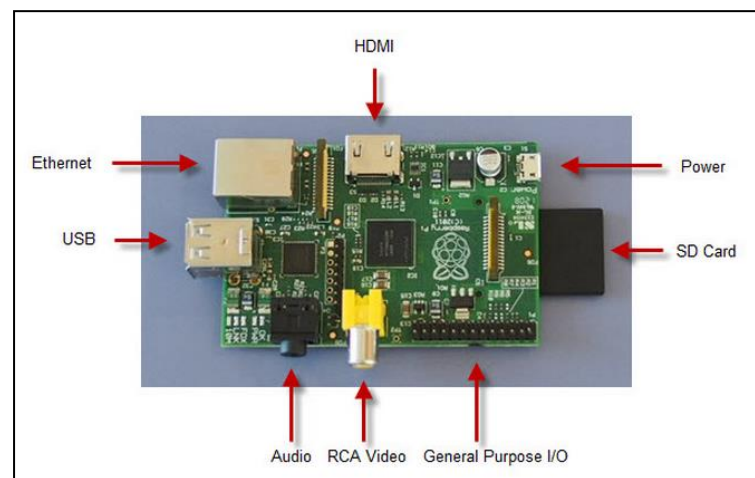
- A differenza di quanto accadeva in C (e C++), **Java e C# nascono dotati da una *infrastruttura standard portabile***
 - indipendenza dalla piattaforma hardware / software
 - write once, run everywhere
- **L'infrastruttura offre servizi per *coprire praticamente ogni esigenza del moderno sviluppo di software***
 - supporto per la grafica (JavaFX, Swing, Java2D, 3D..)
 - programmazione a eventi
 - supporto di rete: URL, Socket, connessioni...
 - supporto per il multi-threading e la concorrenza
 - interfacciamento con database (JDBC)
 - supporto per la sicurezza (cifatura, autenticazione..)
 - esecuzione remota (RMI)...

PORTABILITÀ: JAVA

- L' **infrastruttura Java** è disponibile per praticamente *tutte le piattaforme hardware/software*
- Spesso *embedded* in lettori DVD/BlueRay, TV,...
 - *guarda dietro se trovi il logo!*

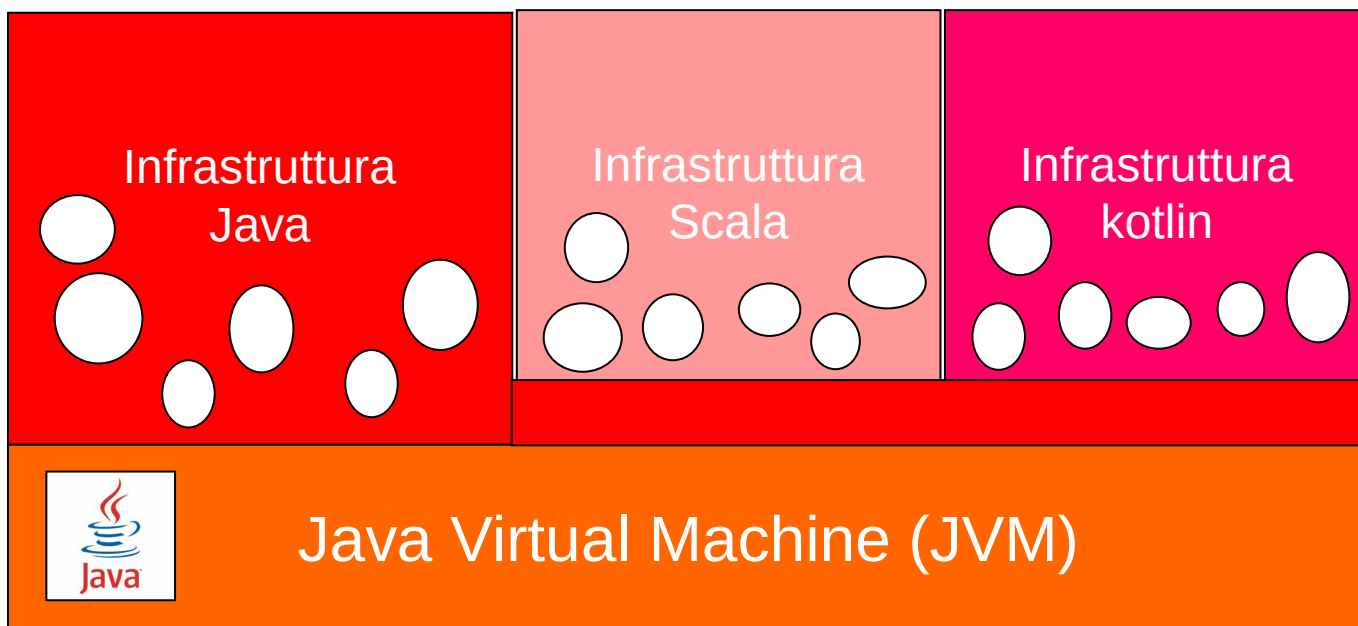


- Disponibile anche su micro-computer low-cost e all-in-one come i Raspberry PI



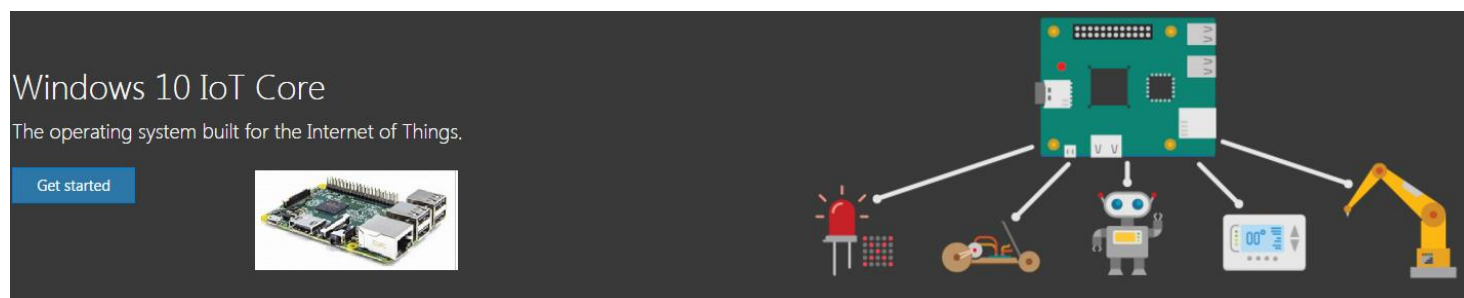
PORTABILITÀ: JAVA

- L' **infrastruttura Java** funge anche da base per molte altre infrastrutture e linguaggi, come **Scala** e **Kotlin**
 - di conseguenza, componenti Java, Scala, Kotlin sono interoperabili



PORTABILITÀ: .NET

- L' **infrastruttura .NET** è disponibile su più piattaforme
 - nativamente, sui sistemi Windows
 - tramite installazione separata per sistemi Linux/Mac
- Molti linguaggi supportati (ma non Java)
- Da Windows10, concetto di *Universal App*
 - unica app per desktop e mobile
- *Windows 10 IoT Core* disponibile per diverse schede (inclusi Raspberry PI)





JAVA e C#: IL SUPPORTO

- Java (e derivati) e C# sono dotati da un *compilatore ottimizzato* che genera codice *per l'infrastruttura*
- A differenza di C e C++, però, qui **il compilatore è solo uno degli strumenti**: il kit include anche strumenti per:
 - produrre *adeguata documentazione* sempre aggiornata
→ **estrazione automatizzata di MANUALI HTML**
 - supportare attività di *collaudo sistematico*
→ **esecuzione automatizzata di SUITE DI TEST**
 - progettare *la distribuzione dell'applicazione (o libreria)*, eventualmente anche con firma digitale (per garantirne autenticità)
→ **impatto sul processo produttivo**
→ **impatto sull'organizzazione dell'applicazione**



IL PROBLEMA DEL DEPLOYMENT

Deployment = distribuzione di un software

- Non può essere lasciata al "fai da te":
anche questa attività va organizzata e progettata
 - è intrinsecamente non scalabile: 10 installazioni sono una cosa, 1000 o 10000 sono tutt'altra faccenda!
- Cosa distribuire?
 - supportare N piattaforme hardware & software *non può significare dover distribuire N eseguibili diversi...* è ingestibile!
 - ideale: distribuire un solo file, per tutte le possibili piattaforme
- Un archivio di *componenti compattati*, simile a uno ZIP
 - magari usabile senza *bisogno neanche di scompattarlo...* 😊
 - per Java e derivati: **formato JAR** (Java ARchive)